claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\agent-a771ff1d85e18f2ed.jsonl`
- SHA-256: `a27185953af0528a41e3e638f02bf9467ff74cfea7a5338174fa5ec1cce9eda5`
- Source modified: `2026-06-15T16:53:11+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `subagents`
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`

Transcript

1. user (2026-06-15T16:35:51.327Z)

You are working in an ISOLATED GIT WORKTREE of the `badminton-highlight-indexer` repo (a local AI badminton rally/highlight tool). Implement ONE focused performance fix and open a PR. Do NOT merge it.

The problem (verified live today)

The WASB shuttle detector is the GPU rally-detection substrate. Its WSL runner (`backend/pipeline/detectors/wasb_runner.py::run_predict`) invokes `backend/pipeline/detectors/wasb_infer.py` which extracts EVERY video frame to disk as PNG via cv2 (`extract_frames`), then runs the GPU detector over them. For a ~12-minute 1080p60 video that's ~46,000 PNGs ? the extraction dominates and a real run exceeded the 1800s (30-min) timeout WITHOUT finishing. This silently produced "0 rallies" (a wasted GPU run). We need WASB frame handling to be FAST.

Your task

1. **Read and understand** `backend/pipeline/detectors/wasb_infer.py` (esp. `extract_frames`, `run`, the resumable detector cache `det_raw.jsonl`/`manifest.json`, and how frames feed the detector + the OnlineTracker) and `backend/pipeline/detectors/wasb_runner.py::run_predict` (how `--video`/`--frames_out_dir` are passed). Also skim `docs/CODE_MAP.md` §2.2 and the memory note about "ffmpeg pre-extract / proxy-index recipe" if referenced in docs.
2. **Choose the lowest-risk speedup that PRESERVES OUTPUT EXACTLY.** This is critical: after this lands, the owner will run a SIDE-BY-SIDE on 1-2 real videos comparing the sped-up path's trajectory CSV against the original cv2 path ? they must be identical (or numerically equivalent). So the detector must see the SAME frames, in the SAME order, at the SAME resolution. Prefer, in order:
   (a) **Stream frames directly from the video** (cv2.VideoCapture read ? feed detector in batches) WITHOUT writing all PNGs to disk, if `run`'s structure allows it ? this eliminates the disk round-trip entirely and is output-preserving.
   (b) If a frames-on-disk contract is hard to remove, make extraction faster (e.g. write JPEG instead of PNG only if it does NOT change detector input ? likely it WOULD change pixels, so avoid unless the detector re-decodes identically; or parallelize extraction).
   Pick ONE approach, justify why it preserves output, and implement it behind a guard so the OLD path remains available (e.g. a config/flag or autodetect) ? zero behavior change unless the fast path is active and equivalent.
   Constraints: **ffmpeg is NOT available inside WSL** (host-only) and `/mnt/c` reads are slow from WSL, so a host-ffmpeg-into-WSL scheme is fragile ? a pure-Python streaming decode inside `wasb_infer` (runs in the WSL conda env which has cv2/torch) is likely the cleanest. Keep the resumable-cache behavior working.
3. **You CANNOT run the GPU/WSL/real WASB here** (worktree, no GPU). So: implement cleanly, and add UNIT TESTS that are mockable/pure (no torch/cv2/GPU needed at collection) ? e.g. test any new pure helper (frame-batching/indexing logic) and that the runner/inference wiring is correct. Mirror the existing mock style in `tests/test_tracknet_runner.py` (they patch `_wsl_bash`, use `subprocess.CompletedProcess`). Do NOT add tests that require a real video/GPU.
4. **Verify locally** (this box has python + cv2/numpy/torch but treat GPU/WSL as unavailable): run `ruff check <changed files>`, `mypy <changed backend files>`, the relevant `pytest

tests/test_*.py` for anything you touched, and `python -m pytest tests/test_import_smoke.py`. Everything you can run must be green.
5. **Open a PR** (do NOT merge):
   - Branch from `origin/master` EXPLICITLY: `git fetch origin && git checkout -b perf/wasb-fast-frames origin/master`. Stage ONLY the files you changed (explicit paths, never `git add -A`). Descriptive commit; end the message with `Co-Authored-By: Claude Fable 5 <noreply@anthropic.com>`.
   - `gh pr create` against base `master`. In the PR body: explain the bottleneck, your chosen approach and WHY it preserves output, and include a clear **"GPU verification + side-by-side plan"** section the owner will follow (run the sped-up path and the original cv2 path on 1-2 real videos, diff the trajectory CSVs / resulting rallies, expect identical).
   - Note in the PR that repo CI is currently red account-wide due to a GitHub Actions BILLING outage (not code) ? so you verified locally.
   - **DO NOT merge the PR** (it needs GPU verification + the side-by-side first). Leave it open.

Guardrails

- One PR, focused on the frame-extraction speedup only. No drive-by refactors.
- Output-preservation is the #1 requirement (the side-by-side must pass).
- Don't touch the running production behavior by default ? gate the fast path so it's safe.
- Report back: the approach you chose, the files changed, test results, and the PR URL.

2. assistant (2026-06-15T16:35:55.156Z)

I'll start by reading the key files to understand the WASB frame-extraction path, then pull the latest master.

3. user (2026-06-15T16:35:58.701Z)

46ad601 fix(detectors): fail fast with an actionable error when a video source is unreadable from WSL (#177)

4. user (2026-06-15T16:35:58.741Z)

```
1       """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3       WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4       own building blocks (ImageDataset transform + detector + online tracker) to run on
5       any folder of extracted frames and emit a trajectory CSV in the exact shape the
6       indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8           Frame,Visibility,X,Y
9
10      It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11      imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12      (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14      Resumable across reboots
15      ------------------------
16      The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17      the slow, expensive part; the CPU tracker pass that turns detections into a
18      trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19      resume mid-stream). We exploit that split:
20
21        * The detector pass writes its per-window raw outputs incrementally to
22          ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23          records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24          most ``--flush-every`` windows of GPU work instead of the whole pass.
25        * On restart we replay the cached windows, skip the DataLoader ahead to the first
26          un-cached window, and continue. Once every window is cached, the detector pass
27          is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28          the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29          not the GPU hour.
30
```

```
    The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
    to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
    weights, sport, window size, or frame count change.

    Usage (in WSL, from ~/models/WASB-SBDT/src):
        python wasb_infer.py --frames_dir /path/frames --weights
../pretrained_weights/wasb_badminton_best.pth.tar \
            --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
every 1000] [--fresh]
    """
    import os
    import os.path as osp
    import csv
    import glob
    import json
    import argparse
    import logging
    from collections import defaultdict

    logger = logging.getLogger(__name__)

    CACHE_VERSION = 1
    _DET_FILE = "det_raw.jsonl"
    _MANIFEST_FILE = "manifest.json"


    def _build_cfg(weights: str, sport: str):
        """Compose the WASB eval config via Hydra, overriding model/weights/device."""
        from hydra import compose, initialize
        with initialize(config_path="configs", version_base=None):
            cfg = compose(config_name="eval", overrides=[
                f"dataset={sport}",
                "model=wasb",
                f"detector.model_path={weights}",
                "runner.device=cuda",
                "runner.gpus=[0]",      # this box has a single GPU; default config assumes
more
            ])
        return cfg


    def _frame_id(path: str) -> int:
        """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
180)."""
        stem = osp.splitext(osp.basename(path))[0]
        digits = "".join(ch for ch in stem if ch.isdigit())
        return int(digits) if digits else -1


    # ------------------------------------------------------------------------ #
    # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
    # ------------------------------------------------------------------------ #
    def _cache_paths(cache_dir: str):
        return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)


    def default_cache_dir(out_csv: str) -> str:
        """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
        d = osp.dirname(osp.abspath(out_csv))
        stem = osp.splitext(osp.basename(out_csv))[0]
        return osp.join(d, stem + "_wasbcache")


    def _atomic_write_text(path: str, text: str) -> None:
        """Write then rename so a crash mid-write can't leave a half-written file."""
```

```
        tmp = path + ".tmp"
        with open(tmp, "w") as f:
            f.write(text)
            f.flush()
            os.fsync(f.fileno())
        os.replace(tmp, path)


    def _det_plain(det) -> list:
        """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
list."""
        xy = det["xy"]
        return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]


    def _dets_to_tracker(plain_list):
        """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
array,
        the tracker does vector math / np.linalg.norm on it)."""
        import numpy as np
        return [{"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
                for p in plain_list]


    def write_manifest(cache_dir: str, manifest: dict) -> None:
        _, mpath = _cache_paths(cache_dir)
        _atomic_write_text(mpath, json.dumps(manifest, indent=2))


    def read_manifest(cache_dir: str):
        _, mpath = _cache_paths(cache_dir)
        if not osp.exists(mpath):
            return None
        try:
            with open(mpath) as f:
                return json.load(f)
        except (json.JSONDecodeError, OSError):
            return None


    def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,
                            frames_in: int, frames_total: int) -> bool:
        """A cache is reusable only if the run that produced it matches this run's
inputs."""
        if not manifest or manifest.get("version") != CACHE_VERSION:
            return False
        return (
            manifest.get("frames_dir") == osp.abspath(frames_dir)
            and manifest.get("weights") == weights
            and manifest.get("sport") == sport
            and manifest.get("frames_in") == frames_in
            and manifest.get("frames_total") == frames_total
        )


    def reset_cache(cache_dir: str) -> None:
        """Drop any existing cache so the next run starts clean."""
        det_jsonl, mpath = _cache_paths(cache_dir)
        for p in (det_jsonl, mpath):
            if osp.exists(p):
                os.remove(p)


    def load_and_clean_cache(cache_dir: str):
        """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
```

```
        rebuild the per-frame detection accumulation, and rewrite the file to exactly that
        prefix so a trailing half-written line from a crash can't corrupt future appends.

        Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
        [x, y, score, scale] candidates (accumulated across every window the frame is in,
        in window order ? identical to a non-resumed run).
        """
        det_jsonl, _ = _cache_paths(cache_dir)
        det_by_fid = defaultdict(list)
        valid: list = []
        if osp.exists(det_jsonl):
            with open(det_jsonl, "r") as f:
                for line in f:
                    s = line.strip()
                    if not s:
                        continue
                    try:
                        obj = json.loads(s)
                    except json.JSONDecodeError:
                        break                       # truncated trailing line (crash mid-
flush)
                    if obj.get("w") != len(valid):  # non-contiguous -> stop at the gap
                        break
                    valid.append(s)
                    for fid, plain in obj["f"]:
                        det_by_fid[int(fid)].extend([list(p) for p in plain])
            # Guarantee a clean append boundary by rewriting only the good prefix.
            _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
        return len(valid), det_by_fid


    def _append_windows(fh, objs) -> None:
        """Append window records as JSONL and durably flush (bounded loss on crash)."""
        for o in objs:
            fh.write(json.dumps(o, separators=(",", ":")) + "\n")
        fh.flush()
        os.fsync(fh.fileno())


    def _atomic_write_trajectory(out_csv: str, rows) -> None:
        os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
        tmp = out_csv + ".tmp"
        with open(tmp, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["Frame", "Visibility", "X", "Y"])
            w.writerows(rows)
            f.flush()
            os.fsync(f.fileno())
        os.replace(tmp, out_csv)


    # ------------------------------------------------------------------------ #
    # Inference
    # ------------------------------------------------------------------------ #
    def run(frames_dir: str, weights: str, out_csv: str, sport: str = "badminton",
            limit: int = 0, batch_size: int = 8, num_workers: int = 4,
            log_every_batches: int = 50, cache_dir: "str | None" = None,
            flush_every: int = 1000, fresh: bool = False) -> str:
        import time
        import torch
        from torch.utils.data import DataLoader
        from detectors import build_detector
        from trackers import build_tracker
        from dataloaders import build_img_transforms, build_seq_transforms
        from dataloaders.dataset_loader import ImageDataset
```

```
        from utils import Center

        cfg = _build_cfg(weights, sport)
        frames_in = int(cfg["model"]["frames_in"])
        input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
        output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))

        frames = sorted(glob.glob(osp.join(frames_dir, "*.png")) +
glob.glob(osp.join(frames_dir, "*.jpg")))
        if limit:
            frames = frames[:limit]
        if len(frames) < frames_in:
            raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in
{frames_dir}")

        # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
checkpoint).
        samples = []
        for i in range(len(frames) - frames_in + 1):
            window = frames[i:i + frames_in]
            annos = [{"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
for p in window]
            samples.append({"frames": window, "annos": annos})
        windows_total = len(samples)

        # ---- cache setup / resume decision ---------------------------------- #
        if cache_dir is None:
            cache_dir = default_cache_dir(out_csv)
        os.makedirs(cache_dir, exist_ok=True)
        det_jsonl, _ = _cache_paths(cache_dir)
        manifest = None if fresh else read_manifest(cache_dir)
        resume = manifest_compatible(
            manifest or {}, frames_dir=frames_dir, weights=weights, sport=sport,
            frames_in=frames_in, frames_total=len(frames),
        )
        if resume:
            windows_done, det_by_fid = load_and_clean_cache(cache_dir)
            logger.info(f"resuming from cache: {windows_done}/{windows_total} windows
already detected")
        else:
            if manifest is not None:
                logger.info("cache incompatible with this run -> starting fresh")
            reset_cache(cache_dir)
            windows_done, det_by_fid = 0, defaultdict(list)

        base_manifest = {
            "version": CACHE_VERSION,
            "frames_dir": osp.abspath(frames_dir),
            "weights": weights,
            "sport": sport,
            "frames_in": frames_in,
            "frames_total": len(frames),
            "windows_total": windows_total,
            "windows_done": windows_done,
        }
        write_manifest(cache_dir, base_manifest)

        # ---- detector pass over the un-cached windows ----------------------- #
        remaining = samples[windows_done:]
        if remaining:
            _, transform_test = build_img_transforms(cfg)
            try:
                _, seq_transform_test = build_seq_transforms(cfg)
            except Exception:
                seq_transform_test = None
```

```
            ds = ImageDataset(cfg, remaining, input_wh, output_wh,
                              transform=transform_test, seq_transform=seq_transform_test,
is_train=False)
            loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
num_workers=num_workers)
            detector = build_detector(cfg)

            t0 = time.time()
            done = windows_done
            win_idx = windows_done
            log_every = max(1, log_every_batches)
            flush_n = max(1, flush_every)
            pending = []
            logger.info(f"detecting on {len(remaining)} remaining windows "
                        f"(total {windows_total}, batch={batch_size},
workers={num_workers})...")
            det_f = open(det_jsonl, "a")
            try:
                with torch.no_grad():
                    for bi, batch in enumerate(loader):
                        imgs, _hms, trans = batch[0], batch[1], batch[2]
                        img_paths = [list(t) for t in batch[-1]]   # [frame_in][batch] ->
path
                        batch_results, _ = detector.run_tensor(imgs, trans)
                        nb = imgs.shape[0]
                        for ib in range(nb):
                            frames_payload = []
                            for ie in sorted(batch_results[ib].keys()):
                                p = img_paths[ie][ib]
                                fid = _frame_id(p)
                                plain = [_det_plain(d) for d in batch_results[ib][ie]]
                                frames_payload.append([fid, plain])
                                det_by_fid[fid].extend(plain)
                            pending.append({"w": win_idx, "f": frames_payload})
                            win_idx += 1
                        done += nb
                        if len(pending) >= flush_n or done >= windows_total:
                            _append_windows(det_f, pending)
                            pending = []
                            base_manifest["windows_done"] = done
                            write_manifest(cache_dir, base_manifest)
                        if (bi + 1) % log_every == 0 or done >= windows_total:
                            el = time.time() - t0
                            new_done = done - windows_done
                            rate = new_done / el if el > 0 else 0.0
                            eta = (windows_total - done) / rate if rate > 0 else 0.0
                            logger.info(f"{done}/{windows_total} "
                                        f"({100 * done // max(1, windows_total)}%) |
{rate:.1f} win/s | "
                                        f"elapsed {el:.0f}s | ETA {eta:.0f}s")
                    if pending:
                        _append_windows(det_f, pending)
                        base_manifest["windows_done"] = done
                        write_manifest(cache_dir, base_manifest)
            finally:
                det_f.close()
            logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
        else:
            logger.info("detector cache already complete -> tracker-only re-run")

        # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
        tracker = build_tracker(cfg)
        tracker.refresh()
        rows = []
        for fid in sorted(det_by_fid.keys()):
```

```
            r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
            rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))

        _atomic_write_trajectory(out_csv, rows)
        visible = sum(1 for _, v, *_ in rows if v)
        logger.info(f"wrote {len(rows)} rows -> {out_csv}")
        logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
        return out_csv


    def extract_frames(video_path: str, frames_dir: str) -> str:
        """Decode a video to PNG frames named by source frame index (00000000.png ...).

        Idempotent: if the dir already holds the expected number of frames (matching the
        container's reported frame count) we skip re-decoding ? re-extraction after a
        reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
        extracting with ffmpeg on the host and passing --frames_dir.
        """
        import cv2
        os.makedirs(frames_dir, exist_ok=True)
        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            raise SystemExit(f"cannot open video: {video_path}")
        expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
        existing = len(glob.glob(osp.join(frames_dir, "*.png")))
        if expected > 0 and existing >= expected:
            cap.release()
            logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
            return frames_dir
        i = 0
        while True:
            ok, frame = cap.read()
            if not ok:
                break
            cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
            i += 1
        cap.release()
        logger.info(f"extracted {i} frames -> {frames_dir}")
        return frames_dir


    def main():
        ap = argparse.ArgumentParser()
        ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
        ap.add_argument("--video", help="video file; frames are extracted internally (in
WSL)")
        ap.add_argument("--frames_out_dir", help="where to extract frames when --video is
used")
        ap.add_argument("--weights", required=True)
        ap.add_argument("--out", required=True)
        ap.add_argument("--sport", default="badminton")
        ap.add_argument("--limit", type=int, default=0, help="cap #frames (for quick
tests)")
        ap.add_argument("--batch-size", type=int, default=8)
        ap.add_argument("--num-workers", type=int, default=4)
        ap.add_argument("--log-every-batches", type=int, default=50, help="progress log
cadence")
        ap.add_argument("--cache-dir", default=None,
                        help="resumable detector cache dir (default: <out>_wasbcache next to
--out)")
        ap.add_argument("--flush-every", type=int, default=1000,
                        help="fsync the detector cache every N windows (bounds reboot
loss)")
        ap.add_argument("--fresh", action="store_true", help="ignore any existing cache and
start over")
```

```
396         args = ap.parse_args()
397
398         frames_dir = args.frames_dir
399         if args.video:
400             frames_dir = extract_frames(args.video, args.frames_out_dir or (args.video +
"_frames"))
401         if not frames_dir:
402             raise SystemExit("provide --frames_dir or --video")
403         run(frames_dir, args.weights, args.out, args.sport, args.limit,
404             batch_size=args.batch_size, num_workers=args.num_workers,
405             log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
406             flush_every=args.flush_every, fresh=args.fresh)
407
408
409     if __name__ == "__main__":
410         main()
411
```

5. assistant (2026-06-15T16:36:01.582Z)

Now let me read the runner and the existing test file to understand the wiring and test style.

6. user (2026-06-15T16:36:02.549Z)

```
1       """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3       Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4         1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5         2. copies wasb_infer.py into the WASB repo's src/ dir,
6         3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7         4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9       So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10      with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11      """
12      import os
13      import shlex
14      import logging
15      import subprocess
16      from dataclasses import dataclass
17      from typing import Dict, Any, Optional, Tuple
18
19      # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20      # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
TrackNetRunner.
21      from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
22      from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
23
24      logger = logging.getLogger(__name__)
25
26      # Windows path to the inference wrapper we stage into WSL.
27      _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
28
29
30      @dataclass
31      class WasbConfig:
32          """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
33          distro: str = "Ubuntu"
34          repo_dir: str = "~/models/WASB-SBDT"
35          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36          conda_env: str = "wasb"
37          weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
```

```
        sport: str = "badminton"
        wsl_stage_dir: str = "~/clips"
        timeout_sec: int = 1800  # 30 min; a hung GPU/conda call is killed (0 = no timeout)
        keep_frames: bool = False  # retain staged video + frame cache after success (debug
only)
        min_free_gb: float = 0.0   # warn if WSL free space below this before a run (0 =
off)

        @classmethod
        def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
            w = (idx_cfg or {}).get("wasb", {}) or {}
            return cls(
                distro=w.get("wsl_distro", "Ubuntu"),
                repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
                conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
                conda_env=w.get("conda_env", "wasb"),
                weights_path=w.get("weights_path",
                                   "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
                sport=w.get("sport", "badminton"),
                wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
                timeout_sec=int(w.get("timeout_sec", 1800)),
                keep_frames=bool(w.get("keep_frames", False)),
                min_free_gb=float(w.get("min_free_gb", 0.0)),
            )


    class WasbRunner(WslCommandMixin, DetectorRunner):
        def __init__(self, cfg: WasbConfig):
            self.cfg = cfg

        def healthcheck(self) -> Tuple[bool, str]:
            """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
            cmd = (f"conda activate {self.cfg.conda_env} && "
                   f"python -c \"import torch; print('torch', torch.__version__, "
                   f"'cuda', torch.cuda.is_available())\"")
            try:
                res = self._wsl_bash(cmd)
            except FileNotFoundError:
                return False, "`wsl` not found ? Windows + WSL2 required."
            except subprocess.TimeoutExpired:
                return False, "WSL healthcheck timed out."
            out = (res.stdout or "").strip()
            if res.returncode != 0:
                return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
            return True, out

        def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
            src_mnt = to_wsl_mnt_path(video_win_path)
            reason = self.wsl_source_unreadable_reason(src_mnt)
            if reason:
                logger.error("Cannot stage video into WSL: %s", reason)
                return None
            dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
            cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
            try:
                res = self._wsl_bash(cmd)
            except subprocess.TimeoutExpired:
                logger.error("Staging video into WSL timed out.")
                return None
            if res.returncode != 0 or "OK" not in (res.stdout or ""):
                logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
                return None
```

```
98              return dest
99
100         def _stage_infer_script(self) -> bool:
101             """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
102             src_mnt = to_wsl_mnt_path(_INFER_WIN)
103             cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
OK"
104             res = self._wsl_bash(cmd)
105             return res.returncode == 0 and "OK" in (res.stdout or "")
106
107         def run_predict(self, video_win_path: str, output_win_dir: str,
108                         video_id: Optional[str] = None) -> Optional[str]:
109             """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
110
111             All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
112             two videos that share a filename (e.g. two ``match.mp4`` from different folders)
113             can never reuse each other's staged frames, resumable cache, or CSV.
114             """
115             # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
116             # translation: to_wsl_mnt_path passes relative paths through unchanged, so
117             # WSL resolved them against the inference cwd (~/models/WASB-SBDT/src) and
118             # the CSV landed inside WSL while Windows polled the repo-relative path.
119             output_win_dir = os.path.abspath(output_win_dir)
120             os.makedirs(output_win_dir, exist_ok=True)
121             # Normalize for cross-platform basename (tests use Windows paths on Linux).
122             video_win_path = str(video_win_path).replace("\\", "/")
123             base = os.path.basename(video_win_path)
124             stem, ext = os.path.splitext(base)
125             # Unique, content-derived key: same filename + different bytes -> different key.
126             key = f"{stem}__{video_id[:12]}" if video_id else stem
127             wsl_out_dir = to_wsl_mnt_path(output_win_dir)
128             expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
129
130             if not self._stage_infer_script():
131                 logger.error("Could not stage wasb_infer.py into WSL.")
132                 return None
133             staged_video = self._stage_video(video_win_path, f"{key}{ext}")
134             if staged_video is None:
135                 return None
136             frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"
137
138             # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the
139             # WSL stage filesystem is already low so the user can clean up before it fills.
140             if self.cfg.min_free_gb > 0:
141                 free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
142                 if free is not None and free < self.cfg.min_free_gb:
143                     logger.warning(
144                         "Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f). Frame
extraction "
145                         "may fill the disk ? consider running tools/cleanup_caches.py.",
146                         free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
147
148             cmd = (
149                 f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
150                 f"python wasb_infer.py "
151                 f"--video {wsl_tilde_quote(staged_video)} "
152                 f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
153                 f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
154                 f"--sport {shlex.quote(self.cfg.sport)} "
155                 f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv')}"
156             )
157             logger.info("Running WASB inference on %s ...", base)
158             try:
159                 res = self._wsl_bash(cmd)
160             except subprocess.TimeoutExpired:
```

```
                logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
                return None
            if res.returncode != 0:
                logger.error("WASB inference failed:\n%s", (res.stderr or
res.stdout)[-2000:])
                return None
            if not os.path.exists(expected_csv_win):
                logger.error("WASB finished but expected CSV not found at %s",
expected_csv_win)
                return None
            logger.info("WASB trajectory CSV: %s", expected_csv_win)

            # Success ? the trajectory CSV is the durable output; the staged video copy and
            # the (huge) decoded frame cache are now pure intermediates. Delete them unless
            # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
            # so the frame cache + detector cache remain for a resumable re-run.
            if not self.cfg.keep_frames:
                logger.info("Cache hygiene: removing staged video + frame cache for %s "
                            "(set indexing.wasb.keep_frames=true to retain).", key)
                self._wsl_rm_rf([frames_dir, staged_video])
            return expected_csv_win

        # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
        parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
        trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)

```

7. user (2026-06-15T16:36:02.914Z)

```
"""Tests for the TrackNet runner's pure-Python pieces (path translation, CSV
parsing, trajectory -> action-window conversion). The WSL subprocess invocation
itself is not exercised here ? these run anywhere, no WSL/TensorFlow needed."""

import os
import subprocess
import textwrap
from unittest.mock import patch

from backend.pipeline.detectors.tracknet_runner import (
    TrackNetRunner,
    TrackNetConfig,
    TrajectoryPoint,
    to_wsl_mnt_path,
)
from backend.pipeline.detectors.base import DetectorRunner
from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig


def _proc(returncode=0, stdout="", stderr=""):
    return subprocess.CompletedProcess(args=[], returncode=returncode,
                                       stdout=stdout, stderr=stderr)


def test_to_wsl_mnt_path_windows():
    assert to_wsl_mnt_path(r"C:\Users\avidu\v.mp4") == "/mnt/c/Users/avidu/v.mp4"
    assert to_wsl_mnt_path(r"D:\clips\m.mp4") == "/mnt/d/clips/m.mp4"


def test_to_wsl_mnt_path_passthrough():
    # Already-POSIX paths are left alone.
    assert to_wsl_mnt_path("/home/avidullu/clips/v.mp4") == "/home/avidullu/clips/v.mp4"
    assert to_wsl_mnt_path("~/clips/v.mp4") == "~/clips/v.mp4"

```

```
35
36      def test_config_from_indexing_cfg():
37          cfg = TrackNetConfig.from_indexing_cfg({"tracknet": {
38              "wsl_distro": "Ubuntu-22.04", "conda_env": "TN", "weights_path": "/w/m.h5",
39              "queue_length": 7, "stage_in_wsl": False,
40          }})
41          assert cfg.distro == "Ubuntu-22.04"
42          assert cfg.conda_env == "TN"
43          assert cfg.weights_path == "/w/m.h5"
44          assert cfg.queue_length == 7
45          assert cfg.stage_in_wsl is False
46
47
48      def test_config_defaults_when_absent():
49          cfg = TrackNetConfig.from_indexing_cfg({})
50          assert cfg.distro == "Ubuntu"
51          assert cfg.conda_env == "TrackNetV4"
52          assert cfg.weights_path == ""  # must be supplied by the user
53
54
55      def test_parse_trajectory_csv(tmp_path):
56          csv_file = tmp_path / "v_predict.csv"
57          csv_file.write_text(textwrap.dedent("""\
58              Frame,Visibility,X,Y
59              0,0,0,0
60              1,1,100,200
61              2,1,140,210
62              3,0,0,0
63          """))
64          pts = TrackNetRunner.parse_trajectory_csv(str(csv_file))
65          assert len(pts) == 4
66          assert pts[0].visible is False
67          assert pts[1].visible is True and pts[1].x == 100 and pts[1].y == 200
68          assert pts[3].visible is False
69
70
71      def test_trajectory_to_action_windows_detects_moving_shuttle():
72          # 30 fps, 1280px wide. Shuttle moves ~60px/frame while visible (frames 0..29) =>
73          # well above the velocity threshold; then disappears. Expect one window ~1s long.
74          fps, width = 30.0, 1280.0
75          pts = [TrajectoryPoint(frame=i, visible=True, x=100 + 60 * i, y=300) for i in
range(30)]
76          pts += [TrajectoryPoint(frame=i, visible=False, x=0, y=0) for i in range(30, 60)]
77
78          windows = TrackNetRunner.trajectory_to_action_windows(
79              pts, fps=fps, frame_width=width, velocity_thresh=0.02,
80              merge_gap=2.0, min_window_duration=0.5, chunk_index=0,
81          )
82          assert len(windows) == 1
83          w = windows[0]
84          assert w["start"] < w["end"]
85          assert w["active_frame_count"] > 0
86          assert w["peak_velocity"] > 0.02
87          assert w["chunk_index"] == 0
88
89
90      def test_trajectory_to_action_windows_ignores_stationary_shuttle():
91          # Visible but barely moving (1px/frame) => below threshold => no windows.
92          pts = [TrajectoryPoint(frame=i, visible=True, x=500 + i, y=300) for i in range(60)]
93          windows = TrackNetRunner.trajectory_to_action_windows(
94              pts, fps=30.0, frame_width=1280.0, velocity_thresh=0.05,
95              merge_gap=2.0, min_window_duration=0.5,
96          )
97          assert windows == []
98
```

```
99
100     def test_trajectory_to_action_windows_splits_on_gap():
101         # Two bursts of motion separated by a long invisible gap => two windows.
102         burst1 = [TrajectoryPoint(frame=i, visible=True, x=100 + 60 * i, y=300) for i in
range(20)]
103         gap = [TrajectoryPoint(frame=i, visible=False, x=0, y=0) for i in range(20, 140)]
104         burst2 = [TrajectoryPoint(frame=i, visible=True, x=100 + 60 * (i - 140), y=300)
105                   for i in range(140, 160)]
106         windows = TrackNetRunner.trajectory_to_action_windows(
107             burst1 + gap + burst2, fps=30.0, frame_width=1280.0,
108             velocity_thresh=0.02, merge_gap=2.0, min_window_duration=0.3,
109         )
110         assert len(windows) == 2
111         assert windows[0]["end"] < windows[1]["start"]
112
113
114     # ------------------------------------------------------------ WSL subprocess glue
115
116     def _runner():
117         return TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5"))
118
119
120     def test_healthcheck_ok():
121         r = _runner()
122         with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="TF 2.17.0 GPUs 1")):
123             ok, msg = r.healthcheck()
124         assert ok is True
125         assert "TF 2.17.0" in msg
126
127
128     def test_healthcheck_failure_returns_message():
129         r = _runner()
130         with patch.object(r, "_wsl_bash", return_value=_proc(1,
stderr="ModuleNotFoundError")):
131             ok, msg = r.healthcheck()
132         assert ok is False
133         assert "ModuleNotFoundError" in msg
134
135
136     def test_healthcheck_no_wsl_binary():
137         r = _runner()
138         with patch.object(r, "_wsl_bash", side_effect=FileNotFoundError()):
139             ok, msg = r.healthcheck()
140         assert ok is False
141         assert "wsl" in msg.lower()
142
143
144     def test_run_predict_requires_weights():
145         r = TrackNetRunner(TrackNetConfig(weights_path=""))  # no weights configured
146         assert r.run_predict(r"C:\v\clip.mp4", "out") is None
147
148
149     def test_run_predict_rejects_bad_extension(tmp_path):
150         r = _runner()
151         assert r.run_predict(r"C:\v\clip.mkv", str(tmp_path)) is None
152
153
154     def test_run_predict_success(tmp_path):
155         r = TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5", stage_in_wsl=False))
156         out_dir = str(tmp_path)
157         # predict.py would write "<stem>_predict.csv"; create it so the existence check
passes.
158         csv_path = os.path.join(out_dir, "clip_predict.csv")
159         with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="Prediction
complete.")):
```

```
            open(csv_path, "w").close()
            result = r.run_predict(r"C:\v\clip.mp4", out_dir)
        assert result == csv_path


    def test_run_predict_failure_returns_none(tmp_path):
        r = TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5", stage_in_wsl=False))
        with patch.object(r, "_wsl_bash", return_value=_proc(1, stderr="CUDA error")):
            assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path)) is None


    def test_run_predict_missing_output_csv(tmp_path):
        """Exit 0 but no CSV produced should be treated as failure."""
        r = TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5", stage_in_wsl=False))
        with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="done")):
            assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path)) is None


    def test_stage_video_success():
        r = _runner()
        # Two WSL calls now: the readability preflight (echo READABLE), then the cp (echo
OK).
        with patch.object(r, "_wsl_bash",
                          side_effect=[_proc(0, stdout="READABLE"), _proc(0, stdout="OK")]):
            staged = r._stage_video(r"C:\v\clip.mp4", "clip.mp4")
        assert staged == "~/clips/clip.mp4"


    def test_stage_video_failure():
        r = _runner()
        # Source readable, but the cp itself fails -> None.
        with patch.object(r, "_wsl_bash",
                          side_effect=[_proc(0, stdout="READABLE"), _proc(1, stderr="disk
full")]):
            assert r._stage_video(r"C:\v\clip.mp4", "clip.mp4") is None


    def test_stage_video_unreadable_source_fails_fast_with_actionable_msg(caplog):
        """A source invisible to WSL (Google Drive / UNC) must fail BEFORE cp, with a clear,
        fixable message ? not a cryptic 'cannot stat' that silently yields 0 rallies."""
        r = _runner()
        import logging
        # Preflight `test -r` returns no READABLE -> unreadable. cp must NOT be attempted.
        with patch.object(r, "_wsl_bash", return_value=_proc(1, stdout="")) as wsl, \
             caplog.at_level(logging.ERROR):
            assert r._stage_video("G:\\My Drive\\Sharing\\clip.mp4", "clip.mp4") is None
        assert wsl.call_count == 1  # only the preflight ran; cp short-circuited
        assert "not readable from WSL" in caplog.text
        assert "local disk" in caplog.text


    def test_wsl_source_unreadable_reason_none_when_readable():
        r = _runner()
        with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="READABLE")):
            assert r.wsl_source_unreadable_reason("/mnt/c/v/clip.mp4") is None


    def test_wsl_source_unreadable_reason_none_on_flaky_probe():
        """A timed-out/raising probe must NOT block a genuinely-readable source."""
        r = _runner()
        with patch.object(r, "_wsl_bash", side_effect=subprocess.TimeoutExpired(cmd="x",
timeout=1)):
            assert r.wsl_source_unreadable_reason("/mnt/c/v/clip.mp4") is None


```

```
222     def test_run_predict_stages_when_enabled(tmp_path):
223         """With stage_in_wsl=True, run_predict should stage the video first."""
224         r = TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5", stage_in_wsl=True))
225         out_dir = str(tmp_path)
226         csv_path = os.path.join(out_dir, "clip_predict.csv")
227         with patch.object(r, "_stage_video", return_value="~/clips/clip.mp4") as mock_stage,
\
228              patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="done")):
229             open(csv_path, "w").close()
230             result = r.run_predict(r"C:\v\clip.mp4", out_dir)
231         mock_stage.assert_called_once()
232         assert result == csv_path
233
234
235     # ------------------------------------------------------------ DetectorRunner ABC
contract
236
237     def test_tracknet_runner_implements_detector_runner_abc():
238         """TrackNetRunner must be a DetectorRunner (for polymorphism / future native
runners)."""
239         r = _runner()
240         assert isinstance(r, DetectorRunner)
241         # healthcheck contract: (bool, str) ? exercised with patch in other tests, but shape
here too
242         with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="TF ok")):
243             res = r.healthcheck()
244         assert isinstance(res, tuple) and len(res) == 2
245         assert isinstance(res[0], bool)
246         assert isinstance(res[1], str)
247
248
249     def test_wasb_runner_implements_detector_runner_abc():
250         """WasbRunner must also satisfy the DetectorRunner contract."""
251         w = WasbRunner(WasbConfig())
252         assert isinstance(w, DetectorRunner)
253         with patch.object(w, "_wsl_bash", return_value=_proc(0, stdout="torch ok")):
254             res = w.healthcheck()
255         assert isinstance(res, tuple) and len(res) == 2
256         assert isinstance(res[0], bool) and isinstance(res[1], str)
257
258
259     # (Intentionally no "no-wsl unpatched predict" test here: the stage/inference paths
260     # raise FileNotFoundError on this Linux box when "wsl" is absent, while only the
261     # healthcheck paths catch it today. The graceful-None cases are already exercised
262     # by the patched tests above and by the hybrid tests. Adding a broad except in
263     # run_predict would be a separate robustness change.)
264
```

8. assistant (2026-06-15T16:36:06.990Z)

Now I understand the architecture well. Let me look at the key detail: how `ImageDataset` consumes frames, to understand exactly what pixels the detector sees. This is critical for output-preservation. Let me also check the CODE_MAP and the memory note about the ffmpeg pre-extract recipe.

9. user (2026-06-15T16:36:09.448Z)

```
df9cf80 ci: add mypy lane (lenient green baseline + explicit quarantine list) (#93)
7fbbeb0 feat(logging): standardize to logging in hot paths + update docs
8615534 refactor: adopt approved layered directory structure
```

10. user (2026-06-15T16:36:10.909Z)

```
76-Legacy `motion`: pixel-diff -> segments + **fabricated** metadata/events (? not ground
```

truth).
77-
78:### 1B. WASB shuttle substrate (P2 of wasb_hybrid)
79-```
80-WasbHybridSegmenter.process_video   @backend/pipeline/segmenters/wasb_hybrid.py
81- -> WasbRunner.healthcheck() (gate; not ok -> return [])
@backend/pipeline/detectors/wasb_runner.py
82- -> WasbRunner.run_predict(video, out_dir)
83:      stage video + wasb_infer.py into WSL fs -> `wsl bash -c 'source conda.sh && python
wasb_infer.py ...'`
84:      -> @backend/pipeline/detectors/wasb_infer.py::run (WSL): sliding windows -> GPU
detector pass (CHECKPOINTED det_raw.jsonl) -> CPU tracker (always full re-run) -> trajectory CSV
85- -> TrackNetRunner.parse_trajectory_csv(csv)
@backend/pipeline/detectors/tracknet_runner.py (shared, re-exported by WasbRunner)
86- -> TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width, chunk_start,
velocity_thresh, merge_gap, min_window_duration, chunk_index) [the model-agnostic WINDOWING
BRAIN]
--
102-Audio probe (diagnostic only, NOT in live pipeline): audio/e1_probe.run
@backend/eval/audio/e1_probe.py
103-```
104:Entrypoints: `@run_eval.py` (single video score), `@scripts/run_phase3_eval.py` (manifest
batch, can segment), `@run_regression.py` (baseline gate, exit 1=regression),
`@backend/eval/calibrate_wasb.py` + `@backend/eval/export_wasb_segments.py` (WASB tuning
drivers).
105-
106----
--
109-
110-### 2.0 Orchestration & config
111:- `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS). Static
UI mounts `/static` from `backend/api/static`. Logging is initialized with `basicConfig` (INFO,
timestamped format); backend modules use `logger = logging.getLogger(__name__)` (replaces stray
prints in hot paths like motion, stitcher, wasb_infer, config loader). User-facing CLI summaries
may still use `print` for direct output.
112- - `::app` FastAPI; `::db_instance`,`::global_config` (`Optional[AppConfig]`) module
globals set ONLY in `main()`. ? importing `app` for ASGI without `main()` -> both `None` ->
every endpoint NPEs.
113-[Omitted long context line]
--
147-- `@backend/pipeline/segmenters/motion.py::MotionPlaySegmenter` (name `'motion'`) ? pixel
absdiff baseline. ? **metadata + serve/smash/foul/winner events are RANDOM**
(random.sample/randint/uniform/choice); only segmenter using `db.add_event` and passing
server/receiver; uses `print()`; honors `max_frames`.
148-
149:### 2.2 Detectors & windowing (WSL-quarantined; ABC abstraction complete)
150-[Omitted long context line]
151-- `@backend/pipeline/detectors/tracknet_runner.py`
--
153- - `::to_wsl_mnt_path` (`C:\x`->`/mnt/c/x`; ? POSIX/`~` passthrough, UNC unhandled),
`::TrajectoryPoint`(frame,visible,x,y).
154-[Omitted long context line]
155:[Omitted long matching line]
156:[Omitted long matching line]
157:[Omitted long matching line]
158-[Omitted long context line]
159-[Omitted long context line]
--
163-- `@backend/eval/annotations.py` ? generic GT loader. `::load_annotations` (`start,end` CSV;
RAISES on bad rows/`start>=end`), `::parse_timestamp` (decimal or MM:SS/HH:MM:SS),
`::write_scaffold`.
164-- `@backend/eval/calibration.py` ? overfit guard. `::cross_validate` (within-video k-fold; ?
defaults `metric='recall'` deliberately), `::leave_one_video_out` (honest cross-video; needs ?2
labelled videos), `::improvement_report` (OPTIMISTIC ? selected on full GT), `::select_best`,
`::kfold_indices`, `::pct_improvement` (? from-zero -> `inf`), `::evaluate`. § `PredictFn =

Callable[[Config], List[Interval]]` = the plug-in seam. ? `generalization_gap >~0.1` = overfit
alarm.
165:- `@backend/eval/calibrate_wasb.py` ? WASB tuning CLI. `::BASELINE=(0.02,2.0,2.0)`,
`::default_grid` (45 cfgs), `::make_predict_fn` (parses trajectory once, closes over
`trajectory_to_action_windows`), `::load_golden_gts` (§`start_time,end_time` cols; ? SILENTLY
skips bad rows ? opposite of annotations.load_annotations). ? `{load_golden_gts,
make_predict_fn, BASELINE}` imported by 4 downstream drivers (calibrate_local,
export_wasb_segments, gemini_refine, audio/e1_probe).
166-- `@backend/eval/calibrate_local.py` ? distill Gemini->local thresholds (windowing + net-
crossing gate, no cloud at runtime). `::local_grid` (180 cfgs), `::make_local_predict_fn` (adds
`rally_gate.apply_gate` when `min_crossings>0`), `::build_videos` (? `SystemExit` if a labels
file yields no rallies), `::run`/`::main`. §
`LocalConfig=(velocity,merge,min_dur,min_crossings)`, `BASELINE_LOCAL=(0.02,2.0,2.0,0)`;
manifest JSON shared with distill_local.
167-- `@backend/eval/export_wasb_segments.py` ? tuned cfg -> golden/slicer CSV + comparison +
optional clip cut. `::TUNED=(0.05,1.0,1.0)` (? from ONE video), `::SEG_FIELDS`,`::COMP_FIELDS`,
`::build_comparison`, `::seconds_to_mmss`, `::_maybe_slice` (lazy-imports rally_slicer).
`--slice` requires `--video`. ? `write_segments_csv` output loads straight into
`rally_slicer.load_rally_labels` AND re-imported by gemini_refine ? schema is load-bearing.
--
172-[Omitted long context line]
173-[Omitted long context line]
174:[Omitted long matching line]
175:[Omitted long matching line]
176-[Omitted long context line]
177:[Omitted long matching line]
178-[Omitted long context line]
179-[Omitted long context line]
180-
181-### 2.3a Audio (E1 probe; NOT adopted for fusion ? ADR-007)
182:[Omitted long matching line]
183-- `@backend/eval/audio/e1_probe.py` ? E1 GO/NO-GO diagnostic (no training/fusion).
`::cluster_hits` (groups thwacks; ? `min_hits>=2` so a lone service-feed hit isn't a rally),
`::run` (discrimination ratio + recall recovery + audio-only P/R/F1), `::main`. reuses
`hit_detector` + `calibrate_wasb.{load_golden_gts,make_predict_fn,BASELINE}` + `metrics`.
184-
--
186-[Omitted long context line]
187-- `@backend/tools/rally_slicer.py` ? standalone CLI. `::compute_clip_windows` (pure, unit-
tested), `::load_rally_labels` (golden schema; ? silently skips degenerate rows),
`::slice_rallies`, `::ClipWindow`(rally:str), `::BEGIN_PADDING=0.5`/`::END_PADDING=1.0` (?
duplicate compiler's by copy). `_read_time -> backend/eval/annotations.parse_timestamp`.
188:[Omitted long matching line]
189-- `@backend/utils/video.py` ? `::get_video_duration` (ffprobe; ? 0.0 on failure =
"unknown"), `::extract_video_chunk(...,reencode=False,scale_width=None)`. ? copy mode cuts at
nearest keyframe (drift) ? pass `reencode=True` for short/accurate clips; `scale_width` requires
`reencode=True`.
190-- `@backend/utils/geometry.py` ? `::get_velocity_normalised` (fraction of frame-width/s; ?
raw-px fallback if width<=0; used by yolo_hybrid), `::calculate_iou` (? +1 px convention
inflates IoU), `::get_center/get_distance/get_velocity`.
--
257----
258-
259:## 4. WASB WSL CONTRACT (external dep `~/models/WASB-SBDT`, MIT; NOT in this repo ? claims
below are unverified against external source, only the repo-side caller wasb_infer.py was
checked)
260:- Env: WSL conda `wasb`, cwd `~/models/WASB-SBDT/src`. Hydra `compose(config_name='eval',
overrides=[dataset=<sport>, model=wasb, detector.model_path=<weights>, runner.device=cuda,
runner.gpus=[0]])`. `gpus=[0]` because box is single-GPU. (overrides VERIFIED in
@backend/pipeline/detectors/wasb_infer.py::_build_cfg.)
261:- `model=wasb` -> `name=hrnet`, `frames_in=3`, `frames_out=3`, input 512x288 (WxH), ImageNet
normalize; affine resize maps heatmap coords -> original-frame coords. (model dims read from
cfg["model"] at runtime in wasb_infer.run.)
262-- `detectors/detector.py::TracknetV2Detector.run_tensor(imgs, affine_mats) -> (results,
hms_vis)`. `results[bid][eid]` = list of `{xy:np.array (ORIGINAL coords), score, scale}`.

(caller uses `detector.run_tensor(imgs, trans)` returning `(batch_results, _)` ? consistent.)
263-- `trackers/online.py::OnlineTracker` ? STATEFUL, sequential per `update()` (assumes every frame fed once, in order, no gaps). `update(frame_dets)->{x,y,visi,score}`; `refresh()` resets. `det['xy']` MUST be np.array. Full ordered re-run is deterministic. (caller calls `build_tracker(cfg)`, `tracker.refresh()`, `tracker.update(...)` returning dict with `visi/x/y` ? consistent.)
264:- `dataloaders/dataset_loader.py::ImageDataset(cfg, samples, input_wh, output_wh, transform, seq_transform, is_train)`; `samples=[{frames:[paths], annos:[{frame_path, center:Center(is_visible,x,y)}]}]`. (caller constructs exactly this ? VERIFIED in wasb_infer.run.)
265-- Weights `pretrained_weights/wasb_badminton_best.pth.tar` ? NOT committed, academic-data-trained -> ? confirm terms before COMMERCIAL deploy. monotrack weights = noncommercial, avoided.
266-
--
285-- **fps / frame_width must be PROBED not assumed:** `_probe_fps_width` / runner fallbacks (30.0, 1280.0) silently mis-scale velocity thresholds; calibrate_wasb CLI bakes `59.94/1920`. The report's `fps_fallback_used` flag surfaces this.
286-- **WSL `/mnt` can't see Google Drive / network mounts;** stage video into native WSL fs (`~/clips`) ? `/mnt/c` reads are slow and Drive-backed paths invisible.
287:- **ffmpeg NOT in WSL** by design: all ffmpeg/ffprobe run Windows-side (must be on PATH); WSL only runs the GPU model. `extract_frames` uses cv2 PNG (slow) ? prefer host ffmpeg + `--frames_dir`. ? Don't import `wasb_infer` from Windows code (it imports torch + WASB repo modules).
288-- **GPU float nondeterminism:** detector pass is checkpointed/resumable but not bit-reproducible; the CPU tracker re-run over the ordered cache IS deterministic.
289-- **`output/` is gitignored:** DB, baselines, cache, reels live there. Predictions MUST exist in `output/<db_name>` before any eval/regression.
--
296-- **`timeout_sec=0` = no timeout:** a hung WSL/GPU call blocks forever.
297-- **Stitch concat assumes identical encode:** per-clip re-encode then `-c copy` concat; if probe fails (0.0) no boundary validation.
298:- **Audio NOT in the live pipeline:** `pipeline/audio/hit_detector` + `eval/audio/e1_probe` are a diagnostic probe (ADR-007); recall 100% but discrimination ~2.2× -> not adopted for fusion.
299-
300----
--
315-| Add / change a report footgun rule | `@backend/reporting/spec.yaml::rules` (declarative; ? `when.path` must match harvest's stage/metric/detail names); facts in `@backend/reporting/harvest.py` |
316-| Tweak what the report records | `@backend/reporting/harvest.py::record_run` + stage builders; emit point is `@backend/main.py` `finally:` -> `@backend/reporting/__init__.py::emit_indexer_report` |
317:| Run calibration (WASB thresholds) | `@backend/eval/calibrate_wasb.py::main` (`--trajectory --labels --fps --frame-width`); trust `cv` recall not `fit` |
318-| Distill Gemini -> local (thresholds / learned) | `@backend/eval/calibrate_local.py::main` (thresholds) OR `@backend/eval/distill_local.py::main` (learned classifier); both `--manifest` |
319:| Refine WASB candidates with Gemini (tuning) | `@backend/eval/gemini_refine.py::main` (`--full-video` skips `--trajectory`; needs `GEMINI_API_KEY`) |
320-| Export tuned rally segments / clips | `@backend/eval/export_wasb_segments.py::main` (`--slice` needs `--video`) |
321-| Export timelines (EDL/JSON/CSV) + basic stats (Value Layer slice) | `backend/tools/export.py` (segments_to_edl/json/csv, basic_stats); called from anywhere that has play_segments (CLI, notebook, future API) |
--
325-| Guard a regression (CI) | `@run_regression.py::main` (`--video-id`); exit 1=regress, 2=missing DB; snapshot via `--update-baseline` |
326-| Run the whole test suite | `@run_all_tests.py` (repo root); or `pytest tests/` directly |
327:| Resume a crashed WASB pass | nothing to do ? `@backend/pipeline/detectors/wasb_infer.py::run` auto-resumes from `det_raw.jsonl`+`manifest.json` next to `--out`; `--fresh` forces restart |
328-| Add a validator | subclass `@backend/pipeline/validators/base.py::VideoValidator`; `registry.register(instance)` in `@backend/pipeline/validators/__init__.py` (mind order; producers before consumers) |
329-| Add a DB column/table | `@backend/infrastructure/database.py::Database._init_db` migration

```
ladder + bump `PRAGMA user_version` (else version-assert tests fail) |
--
347-| `test_wasb_runner.py` | `pipeline/detectors/wasb_runner.WasbRunner` WSL-glue
(DetectorRunner conformance, healthcheck, `parse_trajectory_csv`/`trajectory_to_action_windows`
rebind, run_predict; WSL mocked) |
348-| `test_rally_gate.py` | `pipeline/detectors/rally_gate` (axis/net estimate, crossing count,
gate kills single-toss) |
349:| `test_wasb_cache.py` | `pipeline/detectors/wasb_infer` resumable cache (serialize,
contiguity heal, manifest invalidation, atomic writes, `default_cache_dir`) |
350-| `test_eval_metrics.py` | `eval.metrics` IoU/matching/score/pr_curve |
351-| `test_eval_annotations.py` | `eval.annotations` GT CSV parse/validate, `write_scaffold` |
```

11. assistant (2026-06-15T16:36:18.250Z)

This is very helpful. Key facts confirmed from CODE_MAP §4:
- `ImageDataset(cfg, samples, ...)` where `samples=[{frames:[paths], annos:[{frame_path, center}]}]` ? the dataset reads frames **from disk by path**. The transform applies ImageNet normalize + affine resize to 512x288.
- So `ImageDataset` consumes file paths. To stream from video without writing PNGs, I need to feed decoded frames in-memory instead of paths.

The challenge: `ImageDataset` reads from `frame_path`. To preserve output EXACTLY, the in-memory decoded frame must be **bit-identical** to what `cv2.imwrite(PNG)` + `cv2.imread(PNG)` would produce. PNG is lossless, so `cv2.imread(cv2.imwrite_png(frame))` returns the same array as `frame` (the raw BGR from `cap.read()`). So streaming `cap.read()` frames directly into the dataset transform IS bit-identical to the PNG round-trip.

Let me look at the existing `test_wasb_cache.py` and examine how `ImageDataset.__getitem__` works to understand exactly the decode+transform path I must replicate.

12. user (2026-06-15T16:36:19.630Z)

```
1       """Resumable-cache layer of wasb_infer (pure-Python parts; no WSL/GPU).
2
3       Covers the reboot-resume guarantees: per-window incremental cache, cross-window
4       detection accumulation on replay, crash-truncation recovery, manifest
5       compatibility/invalidation, and atomic trajectory writes.
6       """
7       import json
8       import os
9       import os.path as osp
10
11      import numpy as np
12
13      from backend.pipeline.detectors import wasb_infer as wi
14
15
16      def _win(idx, frames):
17          """A window record: frames = [(fid, [[x,y,score,scale], ...]), ...]."""
18          return {"w": idx, "f": [[fid, dets] for fid, dets in frames]}
19
20
21      def _write_windows(cache_dir, windows):
22          det_jsonl, _ = wi._cache_paths(cache_dir)
23          with open(det_jsonl, "a") as f:
24              wi._append_windows(f, windows)
25
26
27      # ------------------------------------------------------------------------ #
28      # detection (de)serialization
29      # ------------------------------------------------------------------------ #
30      def test_det_plain_and_back_roundtrip():
31          det = {"xy": np.array([439.0, 64.5]), "score": np.float32(0.87), "scale": 0}
32          plain = wi._det_plain(det)
33          assert plain == [439.0, 64.5, float(np.float32(0.87)), 0]
```

```
34
35          back = wi._dets_to_tracker([plain])
36          assert len(back) == 1
37          assert isinstance(back[0]["xy"], np.ndarray)        # tracker does vector math on xy
38          assert back[0]["xy"].tolist() == [439.0, 64.5]
39          assert back[0]["scale"] == 0
40
41
42      # ---------------------------------------------------------------------- #
43      # cache replay + resume index
44      # ---------------------------------------------------------------------- #
45      def test_replay_accumulates_detections_across_windows(tmp_path):
46          cache = str(tmp_path / "c")
47          os.makedirs(cache)
48          # frame 1 appears in BOTH windows -> its candidates must accumulate in window order;
49          # frame 2 is empty in window 0 but detected in window 1.
50          _write_windows(cache, [
51              _win(0, [(0, [[10, 20, 0.9, 0]]), (1, [[11, 21, 0.8, 0]]), (2, [])]),
52              _win(1, [(1, [[12, 22, 0.7, 0]]), (2, [[13, 23, 0.6, 0]]), (3, [])]),
53          ])
54          windows_done, det = wi.load_and_clean_cache(cache)
55          assert windows_done == 2
56          assert det[0] == [[10, 20, 0.9, 0]]
57          assert det[1] == [[11, 21, 0.8, 0], [12, 22, 0.7, 0]]   # window-0 then window-1
58          assert det[2] == [[13, 23, 0.6, 0]]
59          assert det[3] == []                                     # seen but never detected
60
61
62      def test_resume_index_equals_window_count(tmp_path):
63          cache = str(tmp_path / "c")
64          os.makedirs(cache)
65          _write_windows(cache, [_win(i, [(i, [[i, i, 0.5, 0]])]) for i in range(5)])
66          windows_done, _ = wi.load_and_clean_cache(cache)
67          assert windows_done == 5
68
69
70      def test_empty_cache(tmp_path):
71          cache = str(tmp_path / "c")
72          os.makedirs(cache)
73          windows_done, det = wi.load_and_clean_cache(cache)
74          assert windows_done == 0 and dict(det) == {}
75
76
77      # ---------------------------------------------------------------------- #
78      # crash robustness
79      # ---------------------------------------------------------------------- #
80      def test_truncated_trailing_line_recovered_and_cleaned(tmp_path):
81          cache = str(tmp_path / "c")
82          os.makedirs(cache)
83          _write_windows(cache, [_win(0, [(0, [[1, 1, 0.9, 0]])]),
84                                 _win(1, [(1, [[2, 2, 0.8, 0]])])])
85          det_jsonl, _ = wi._cache_paths(cache)
86          with open(det_jsonl, "a") as f:                 # simulate a crash mid-flush
87              f.write('{"w":2,"f":[[2,[[3,3,')
88          windows_done, det = wi.load_and_clean_cache(cache)
89          assert windows_done == 2                         # only the two clean windows
survive
90          assert det[1] == [[2, 2, 0.8, 0]]
91          # file is rewritten to the clean prefix -> appending window 2 stays contiguous
92          with open(det_jsonl) as f:
93              lines = [json.loads(ln) for ln in f if ln.strip()]
94          assert [o["w"] for o in lines] == [0, 1]
95          _write_windows(cache, [_win(2, [(2, [[3, 3, 0.7, 0]])])])
96          assert wi.load_and_clean_cache(cache)[0] == 3
97
```

```

    def test_noncontiguous_window_index_stops_replay(tmp_path):
        cache = str(tmp_path / "c")
        os.makedirs(cache)
        _write_windows(cache, [_win(0, [(0, [[1, 1, 0.9, 0]])]),
                               _win(5, [(5, [[2, 2, 0.8, 0]])])])   # gap -> stop after w=0
        windows_done, det = wi.load_and_clean_cache(cache)
        assert windows_done == 1
        assert 5 not in det


    # ------------------------------------------------------------------------ #
    # manifest: atomic write + compatibility / invalidation
    # ------------------------------------------------------------------------ #
    def test_manifest_roundtrip_and_compat(tmp_path):
        cache = str(tmp_path / "c")
        os.makedirs(cache)
        m = {
            "version": wi.CACHE_VERSION,
            "frames_dir": osp.abspath("/x/frames"),
            "weights": "w.pth.tar",
            "sport": "badminton",
            "frames_in": 3,
            "frames_total": 100,
            "windows_total": 98,
            "windows_done": 42,
        }
        wi.write_manifest(cache, m)
        assert not osp.exists(osp.join(cache, wi._MANIFEST_FILE + ".tmp"))   # atomic, no
tmp left
        assert wi.read_manifest(cache) == m

        base = dict(frames_dir="/x/frames", weights="w.pth.tar", sport="badminton",
                    frames_in=3, frames_total=100)
        assert wi.manifest_compatible(m, **base)
        assert not wi.manifest_compatible(m, **{**base, "frames_total": 101})   # more
frames
        assert not wi.manifest_compatible(m, **{**base, "weights": "other.pth"})
        assert not wi.manifest_compatible(m, **{**base, "sport": "tennis"})
        assert not wi.manifest_compatible({}, **base)                           # no cache
        assert not wi.manifest_compatible({**m, "version": 999}, **base)       # schema bump


    def test_read_manifest_missing_returns_none(tmp_path):
        assert wi.read_manifest(str(tmp_path)) is None


    def test_reset_cache_removes_files(tmp_path):
        cache = str(tmp_path / "c")
        os.makedirs(cache)
        _write_windows(cache, [_win(0, [(0, [[1, 1, 0.9, 0]])])])
        wi.write_manifest(cache, {"version": wi.CACHE_VERSION})
        wi.reset_cache(cache)
        det_jsonl, mpath = wi._cache_paths(cache)
        assert not osp.exists(det_jsonl) and not osp.exists(mpath)


    # ------------------------------------------------------------------------ #
    # misc helpers
    # ------------------------------------------------------------------------ #
    def test_default_cache_dir_is_next_to_output():
        out = osp.join("some", "dir", "sample_long_traj.csv")
        cd = wi.default_cache_dir(out)
        assert osp.basename(cd) == "sample_long_traj_wasbcache"
        assert osp.dirname(cd) == osp.dirname(osp.abspath(out))
```

```
161
162
163     def test_atomic_write_trajectory(tmp_path):
164         out = str(tmp_path / "out" / "traj.csv")
165         wi._atomic_write_trajectory(out, [(0, 1, 439.0, 64.0), (1, 0, -1, -1)])
166         assert not osp.exists(out + ".tmp")
167         with open(out) as f:
168             text = f.read().splitlines()
169         assert text[0] == "Frame,Visibility,X,Y"
170         assert text[1] == "0,1,439.0,64.0"
171         assert text[2] == "1,0,-1,-1"
172
```

13. assistant (2026-06-15T16:36:33.788Z)

Now I have a thorough understanding. Let me design the approach carefully.

Analysis & approach

The bottleneck: `extract_frames` decodes every frame via `cv2.VideoCapture.read()` and writes each as a PNG via `cv2.imwrite`. PNG encoding is CPU-heavy (zlib compression of ~6MB per 1080p frame). For ~46k frames this dominates and overruns the 30-min timeout. Then `run()` re-reads those PNGs via `ImageDataset`.

Key insight for output-preservation: `ImageDataset.__getitem__` does `cv2.imread(frame_path)` ? BGR uint8 array ? transform (affine resize to 512x288 + ImageNet normalize). PNG is **lossless**, so `cv2.imread(written_png)` returns the *exact same* uint8 BGR array that `cap.read()` produced. Therefore, if I feed the in-memory `cap.read()` frame directly into the same transform pipeline, the tensor the detector sees is bit-identical to the PNG round-trip. **Output is preserved exactly.**

Chosen approach (option a ? stream from video): Add a streaming path that decodes the video with `cv2.VideoCapture` and feeds frames directly into a custom in-memory `Dataset` that mirrors `ImageDataset`'s per-sample logic, eliminating the PNG disk round-trip entirely. This is gated behind a `--stream-video` flag (and a config field), so the existing frames-on-disk path remains the default until the owner's side-by-side confirms equivalence.

The risk with reproducing `ImageDataset.__getitem__` is that I'd be reimplementing external WASB code I can't see here, which threatens exact-equivalence. Let me instead **reuse `ImageDataset` itself** but supply it in-memory frames. Let me check whether `ImageDataset` can be driven from preloaded arrays, or if I should subclass it.

Since I cannot see the WASB external source, the safest output-preserving design is: **keep `ImageDataset` exactly as-is reading from paths, but eliminate the PNG bottleneck by writing the frames `ImageDataset` reads in a cheaper-yet-lossless form is risky (JPEG changes pixels). Instead, the cleanest output-preserving streaming approach is to subclass `ImageDataset` and override only the per-frame *image loading* to pull from an in-memory decoded-video buffer keyed by frame path**, leaving every transform/affine/normalize step untouched.

Let me check how `ImageDataset.__getitem__` loads images to find the right override seam.

14. user (2026-06-15T16:36:35.336Z)

```
---
no external WASB source available in this worktree (expected)
```

15. assistant (2026-06-15T16:36:52.045Z)

As expected, the external WASB source isn't available here. Since I can't see `ImageDataset.__getitem__`, reproducing its exact image-loading is risky.

The most robust output-preserving design that does NOT require me to reverse-engineer unseen external code: **decode the video once with `cv2.VideoCapture` and write the PNGs (lossless, same bytes) ? but eliminate the slowness by NOT writing PNGs at all when streaming, and instead

drive `ImageDataset` from an in-memory frame store via a thin subclass that only overrides image loading.**

But the override seam is unknown. Let me reconsider. The task's preferred option (a) is "stream frames directly from the video ? feed detector in batches WITHOUT writing all PNGs". The detector's input pipeline is `ImageDataset` ? transform ? `detector.run_tensor(imgs, trans)`.

The cleanest seam I *can* control without seeing external code: the `frames_dir` is decoded to PNGs by *my* `extract_frames`. I can make a streaming `ImageDataset` subclass that overrides `__getitem__`'s file read. But I need to know what `__getitem__` returns and how it reads. Since that's external and unseen, subclassing is fragile.

Safest output-preserving win that I can fully verify: The PNG write is the bottleneck, and PNG decode-on-read is *also* part of it. The truly safe approach that doesn't touch the unseen `ImageDataset` internals at all: **build an in-memory `Dataset` for the detector that exactly mirrors the documented `ImageDataset` contract using the SAME transform objects that `run()` already builds** (`build_img_transforms`, `build_seq_transforms`) and the SAME `Center` annotation shape ? feeding it `cv2`-decoded frames from a single streaming pass.

The problem remains: I don't know the exact ordering of operations inside `ImageDataset.__getitem__` (BGR vs RGB conversion, channel order, affine matrix computation). Reproducing it risks divergence.

Given the constraints and the #1 requirement (exact output preservation, owner will diff CSVs), the **lowest-risk** option is actually a refinement of option (a) that keeps `ImageDataset` 100% untouched: **subclass `ImageDataset`, override ONLY the lowest-level frame-bytes loader to read from an in-memory dict instead of disk.** WASB's `ImageDataset` (per CODE_MAP and standard TrackNet/WASB code) reads each frame via `cv2.imread(frame_path)`. I'll patch that single call site by monkeypatching `cv2.imread` *within a context* to resolve from an in-memory store ? that way every other line of `__getitem__` runs identically, guaranteeing bit-identical tensors.

Let me verify what `cv2.imread` returns vs `cap.read()`: both return BGR uint8 `ndarray`. PNG round-trip is lossless ? identical. This is provably output-preserving and requires zero knowledge of `__getitem__` internals. Let me implement this.

16. user (2026-06-15T16:36:53.024Z)

Chapter marked: "WASB fast-frames implementation". Continue your current work.

17. assistant (2026-06-15T16:37:04.692Z)

Let me reconsider the monkeypatch-`cv2.imread` approach more carefully for correctness and decide the cleanest implementation. There's a subtlety: with `num_workers > 0`, the DataLoader spawns worker processes that won't share the in-memory frame store or the monkeypatch. So streaming requires `num_workers=0` (single process). That's fine and even simpler ? and I'll enforce it in the streaming path.

Also, the in-memory store of ~46k decoded 1080p frames would be ~280GB ? too large to hold all at once. So I cannot preload all frames. I need **windowed streaming**: decode the video sequentially, holding only a sliding buffer of frames currently needed. Since windows are processed in order (`shuffle=False`) and each window of `frames_in=3` consecutive frames slides by 1, I only need a small buffer.

This makes a custom in-memory `Dataset` the right structure: with `shuffle=False` and `num_workers=0`, `__getitem__` is called for sample index 0, 1, 2, ? in order. Sample `i` needs frames `[i, i+1, i+2]`. So a `VideoCapture` read sequentially with a tiny ring buffer covers it.

The cleanest, fully output-preserving design: a `_StreamingImageDataset(ImageDataset)` subclass that:
1. Holds a `cv2.VideoCapture` and a small frame cache keyed by frame index.
2. Overrides only image acquisition: when `ImageDataset.__getitem__` would `cv2.imread(path)`, it instead returns the decoded frame for that index from the streaming buffer.

To override "only image acquisition" without seeing internals, I'll wrap the `__getitem__` call with a `cv2.imread` monkeypatch scoped to the streaming dataset that maps `frame_path` ? the

decoded array. The synthetic frame paths I generate encode the frame index (`{i:08d}.png`), so `imread(path)` ? parse index ? return buffered frame. Let me build this now.

18. user (2026-06-15T16:37:06.196Z)

```
1       """Abstract base for detector runners (TrackNet, WASB, future native, etc.).
2
3       This completes the Detector Runner Abstraction (low-regret item 6 continuation / item 1
4       of the post-observability re-prioritization) to de-risk the WSL/conda dependency for
5       the high-accuracy shuttle trajectory substrate used by tracknet_hybrid / wasb_hybrid.
6
7       The ABC lets us:
8       - Keep the current WSL adapters working unchanged for Windows users.
9       - Swap in (or A/B test) a Linux-native implementation later with zero changes to
10        the hybrid segmenters or the model-agnostic windowing brain.
11      - Provide a single place to evolve the healthcheck / error contract and thread
12        detector readiness into per-video reports.
13
14      Contract (deliberately matches what the concrete runners + hybrids + tests already did):
15      - run_predict(video_win_path: str, output_win_dir: str) -> Optional[str]
16        Run detector, return the Windows-side path to the produced trajectory CSV
17        (e.g. ".../clip_predict.csv" or ".../clip_wasb.csv"), or None on any failure.
18        The caller (hybrid) is responsible for output dir; impls may stage the video
19        into WSL fs for perf.
20
21      - healthcheck() -> tuple[bool, str]
22        Return (ok, message). Never raises. "ok" decides whether to proceed to run_predict.
23        Message is logged (and can be surfaced to reports or UX in future).
24        Intended to be cheap (env existence, import, GPU visibility, weights present).
25
26      Existing behavior is 100% preserved. The only additions are the inheritance and
27      the explicit common type for future implementers.
28
29      How to add a new runner (for docs / future contributors):
30      1. Subclass DetectorRunner in a new or existing module under pipeline/detectors/.
31      2. Implement run_predict and healthcheck (return (bool, str) tuple).
32      3. If it produces the same CSV schema (Frame,Visibility,X,Y), reuse or re-export
33         TrackNetRunner.parse_trajectory_csv and .trajectory_to_action_windows (they are
34         deliberately static and model-agnostic). If the new detector needs different
35         windowing, it can implement its own or we generalize later.
36      4. Lazy-import heavy native deps inside the methods (so importing the segmenter
37         registry never pulls torch/tf on a machine that doesn't have them).
38      5. Update the two hybrid segmenters (or introduce a detector= choice) to accept
39         DetectorRunner instances (they already do the right thing via duck-typing + the
40         type hints we add here).
41      6. Add unit tests that exercise the contract (isinstance, healthcheck shape, run
42         returns str|None) using mocks for the external bits ? see test_tracknet_runner.py.
43      7. No change to eval/calibration code paths (they consume the CSV + the static
44         windowing helpers directly).
45
46      Optional Linux-native path (the main de-risk goal):
47      A LinuxNativeRunner (or ONNXRunner) would:
48      - Skip all wsl / _stage / conda activate.
49      - Load weights locally (torch or onnx) or call a native binary.
50      - Write the exact same CSV format so trajectory_to_action_windows works verbatim.
51      - healthcheck can just check "weights file exists and torch importable + CUDA?".
52      This removes the WSL requirement for Linux users / future SaaS workers while
53      keeping the exact same candidate quality characteristics (or better, once native
54      weights are available).
55
56      Error surfacing into reports:
57      The healthcheck message is already logged by the hybrids on failure. The reporting
58      harvest sees the outcome as "0 candidates" + detector name. When we wire richer
59      segmenter Results (or pass detector_health into emit_indexer_report context), the
60      (ok, msg) can be recorded explicitly under segmentation.details or a new detector
```

```
    stage. The ABC makes that future change local to one place.

    See also:
    - backend/pipeline/segmenters/{tracknet_hybrid,wasb_hybrid}.py (the two callers)
    - tests/test_tracknet_runner.py (contract + pure logic tests; hybrids mock the runner)
    - docs/TRACKNET_WSL_SETUP.md and CODE_MAP.md (2.2 Detectors section)
    """

    import logging
    import shlex
    import subprocess
    from abc import ABC, abstractmethod
    from typing import Any, Callable, List, Optional, Tuple

    logger = logging.getLogger(__name__)


    def wsl_tilde_quote(path: str) -> str:
        """shlex.quote that still lets bash expand a leading ``~/``.

        ``shlex.quote("~/clips/x.mp4")`` single-quotes the whole path, so bash
        treats ``~`` literally and ``cp``/``mkdir`` fail with "No such file or
        directory" ? this broke live WSL staging for the default ``~/clips``
        stage dir (found by the 2026-06-11 wasb_hybrid smoke run; the BACKLOG
        "shlex-quote breaks ~ expansion" item). Quoting only the part after
        ``~/`` keeps expansion AND protects spaces/specials in the remainder.
        """
        if path == "~":
            return path
        if path.startswith("~/"):
            return "~/" + shlex.quote(path[2:])
        return shlex.quote(path)


    class WslCommandMixin:
        """Shared ``wsl -d <distro> bash -c 'source <conda_sh> && <cmd>'`` invocation.

        Lives beside (not on) DetectorRunner: a future Linux-native runner must not
        inherit WSL plumbing. Requires ``self.cfg`` with ``distro``, ``conda_sh``
        and ``timeout_sec`` attributes (TrackNetConfig and WasbConfig both qualify).
        ``timeout_sec == 0`` maps to None = wait forever (configs default 1800s).
        """

        cfg: Any

        def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
            full = f"source {self.cfg.conda_sh} && {inner_cmd}"
            argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
            logger.debug("WSL exec: %s", full)
            return subprocess.run(argv, capture_output=True, text=True,
                                  timeout=(self.cfg.timeout_sec or None))

        # ---- cache hygiene helpers (shared by both WSL runners) ------------------
        def _wsl_free_gb(self, wsl_path: str) -> Optional[float]:
            """Best-effort free space (GB) on the WSL filesystem holding ``wsl_path``.

            Returns None if it can't be determined (never raises). Used as a pre-run
            disk guard so a long extraction doesn't fill the disk mid-flight.
            """
            try:
                res = self._wsl_bash(f"df -P -B1 {wsl_tilde_quote(wsl_path)} | awk
'NR==2{{print $4}}'")
            except (subprocess.TimeoutExpired, OSError):
                return None
            try:
```

```
125                 return int((res.stdout or "").strip()) / (1024 ** 3)
126             except (ValueError, AttributeError):
127                 return None
128 
129         def _wsl_rm_rf(self, wsl_paths: List[str]) -> None:
130             """Best-effort recursive delete of WSL paths (post-success cleanup; never
raises)."""
131             paths = [p for p in wsl_paths if p]
132             if not paths:
133                 return
134             quoted = " ".join(wsl_tilde_quote(p) for p in paths)
135             try:
136                 self._wsl_bash(f"rm -rf {quoted}")
137             except (subprocess.TimeoutExpired, OSError) as e:
138                 logger.warning("WSL cache cleanup failed (non-fatal): %s", e)
139 
140         def wsl_source_unreadable_reason(self, src_mnt: str) -> Optional[str]:
141             """Actionable message if ``src_mnt`` is NOT readable from inside WSL, else None.
142 
143             Google Drive (``G:``/Drive File Stream), UNC and many network/virtual
144             filesystems are invisible to WSL's ``/mnt`` ? staging a video from such a path
145             fails with a cryptic ``cp: cannot stat`` and the whole run silently produces no
146             trajectory (i.e. "0 rallies", a wasted run that looks like a quality result).
147             A cheap ``test -r`` preflight turns that into one clear, fixable error BEFORE
any
148             frame extraction. Best-effort: a flaky/timed-out probe returns None so the real
149             ``cp`` still gets its chance (we never block a genuinely-readable source).
150             """
151             try:
152                 res = self._wsl_bash(f"test -r {shlex.quote(src_mnt)} && echo READABLE")
153             except (subprocess.TimeoutExpired, OSError):
154                 return None
155             if "READABLE" in (res.stdout or ""):
156                 return None
157             return (
158                 f"video source is not readable from WSL at '{src_mnt}'. Google Drive, UNC, "
159                 f"and other network/virtual filesystems are invisible to WSL's /mnt ? copy "
160                 f"the video to a local disk (e.g. C:\\ or the WSL filesystem) and retry."
161             )
162 
163 
164     class DetectorRunner(ABC):
165         """Common interface for shuttle/ball trajectory detectors (WSL or otherwise)."""
166 
167         @abstractmethod
168         def run_predict(self, video_win_path: str, output_win_dir: str,
169                         video_id: Optional[str] = None) -> Optional[str]:
170             """Run inference and return path to trajectory CSV (or None on failure).
171 
172             ``video_id`` (the file MD5) namespaces all staged/cached/output artifacts so two
173             videos that happen to share a filename cannot reuse each other's frames or CSV.
174             It is optional for back-compat; callers in the live pipeline always pass it.
175             """
176             pass
177 
178         @abstractmethod
179         def healthcheck(self) -> Tuple[bool, str]:
180             """Quick check if the runner (env, weights, etc.) is usable.
181 
182             Returns (ok, message). Never raises. Cheap to call before a long run.
183             """
184             pass
185 
186         # De facto trajectory contract: the trajectory hybrids call these
187         # unconditionally after run_predict. Concrete runners reuse/re-export the
```

```
188         # model-agnostic TrackNetRunner staticmethods (same CSV schema + windowing);
189         # a runner with custom windowing supplies its own with the same shapes.
190         parse_trajectory_csv: Callable[..., Any]
191         trajectory_to_action_windows: Callable[..., Any]
192
```

19. assistant (2026-06-15T16:37:21.124Z)

Now I have everything I need. Let me design the implementation precisely.

Implementation plan

In `wasb_infer.py`:

1. **A pure helper `_index_from_frame_path(path)`** ? already exists as `_frame_id`. I'll reuse it. The streaming dataset generates synthetic frame paths `{i:08d}.png` so the detector's `_frame_id` parsing AND the dataset's resolution both work off the same index.

2. **`StreamingFrameReader`** ? a class wrapping `cv2.VideoCapture` that yields frames in index order with a tiny ring buffer, supporting the in-order, possibly-revisited (windows overlap by `frames_in-1`) access pattern. Since `ImageDataset` with `shuffle=False`, `num_workers=0` requests sample `i` (frames `i..i+frames_in-1`) in increasing `i`, the max look-back is `frames_in-1`. I'll implement a sequential reader with a small dict buffer that decodes forward to the requested index and evicts indices below the lowest still-needed one.

3. **`run_streaming(...)`** ? mirrors `run()` but:
   - Computes `frames_total` from the video (`CAP_PROP_FRAME_COUNT`), generates synthetic frame paths.
   - Builds the SAME windows/samples list (synthetic paths).
   - Uses the SAME cache logic (manifest keyed on a `video:<path>` sentinel instead of frames_dir; `frames_total` from probe).
   - Wraps the DataLoader iteration with a scoped `cv2.imread` patch that resolves synthetic paths ? in-memory decoded frames from the streaming reader.
   - Forces `num_workers=0` (in-memory store + monkeypatch only valid in-process).

Actually, to maximize code reuse and minimize divergence risk, the cleanest is: **keep `run()` as the single inference body, and make it accept an optional `frame_loader` callable.** When `frame_loader` is provided (streaming mode), patch `cv2.imread` to delegate to it during the DataLoader pass and force `num_workers=0`. This keeps ALL the windowing/cache/tracker logic in one place ? zero behavioral divergence between disk and stream paths except the source of pixels.

Let me factor the pure pieces so they're unit-testable. I'll add:
- `frame_paths_for_count(n)` ? pure: synthetic ordered paths for `n` frames (`["00000000.png", ...]`).
- `build_windows(frames, frames_in)` ? pure: the sliding-window `samples` builder (extracted from `run`).
- `SequentialFrameStore` ? the ring-buffer logic, unit-testable with a fake reader (no cv2 needed).

Let me write the implementation.

20. user (2026-06-15T16:37:23.173Z)

```
204     # ---------------------------------------------------------------------------- #
205     # Inference
206     # ---------------------------------------------------------------------------- #
207     def run(frames_dir: str, weights: str, out_csv: str, sport: str = "badminton",
208             limit: int = 0, batch_size: int = 8, num_workers: int = 4,
209             log_every_batches: int = 50, cache_dir: "str | None" = None,
210             flush_every: int = 1000, fresh: bool = False) -> str:
211         import time
212         import torch
213         from torch.utils.data import DataLoader
```

```
214         from detectors import build_detector
215         from trackers import build_tracker
216         from dataloaders import build_img_transforms, build_seq_transforms
217         from dataloaders.dataset_loader import ImageDataset
218         from utils import Center
219 
220         cfg = _build_cfg(weights, sport)
221         frames_in = int(cfg["model"]["frames_in"])
222         input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
223         output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
224 
225         frames = sorted(glob.glob(osp.join(frames_dir, "*.png")) +
glob.glob(osp.join(frames_dir, "*.jpg")))
226         if limit:
227             frames = frames[:limit]
228         if len(frames) < frames_in:
229             raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in
{frames_dir}")
230 
231         # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
checkpoint).
232         samples = []
233         for i in range(len(frames) - frames_in + 1):
234             window = frames[i:i + frames_in]
235             annos = [{"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
for p in window]
236             samples.append({"frames": window, "annos": annos})
237         windows_total = len(samples)
238 
239         # ---- cache setup / resume decision ---------------------------------- #
240         if cache_dir is None:
241             cache_dir = default_cache_dir(out_csv)
242         os.makedirs(cache_dir, exist_ok=True)
243         det_jsonl, _ = _cache_paths(cache_dir)
244         manifest = None if fresh else read_manifest(cache_dir)
245         resume = manifest_compatible(
246             manifest or {}, frames_dir=frames_dir, weights=weights, sport=sport,
247             frames_in=frames_in, frames_total=len(frames),
248         )
249         if resume:
250             windows_done, det_by_fid = load_and_clean_cache(cache_dir)
251             logger.info(f"resuming from cache: {windows_done}/{windows_total} windows
already detected")
252         else:
253             if manifest is not None:
254                 logger.info("cache incompatible with this run -> starting fresh")
255             reset_cache(cache_dir)
256             windows_done, det_by_fid = 0, defaultdict(list)
257 
258         base_manifest = {
259             "version": CACHE_VERSION,
260             "frames_dir": osp.abspath(frames_dir),
261             "weights": weights,
262             "sport": sport,
263             "frames_in": frames_in,
264             "frames_total": len(frames),
265             "windows_total": windows_total,
266             "windows_done": windows_done,
267         }
268         write_manifest(cache_dir, base_manifest)
269 
```

21. assistant (2026-06-15T16:37:27.225Z)

Now I'll implement the changes. First, let me add the pure helpers and the streaming frame store

after the existing pure helpers section, then refactor `run()` to accept an optional frame source, and update `extract_frames`/`main()`/the runner.

Let me start by adding pure helpers (synthetic frame paths, window builder, sequential frame store) right before the Inference section.

22. user (2026-06-15T16:38:02.087Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors\wasb_infer.py has been updated successfully. (file state is current in your context ? no need to Read it back)

23. assistant (2026-06-15T16:38:05.006Z)

Now I need to update the manifest's `frames_dir` field to use `cache_key`, and wire the `frame_loader` into the DataLoader pass. Let me view the current state of the relevant section.

24. user (2026-06-15T16:38:06.303Z)

```
330         samples = []
331         for window in build_windows(frames, frames_in):
332             annos = [{"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
for p in window]
333             samples.append({"frames": window, "annos": annos})
334         windows_total = len(samples)
335
336         # ---- cache setup / resume decision ---------------------------------- #
337         if cache_dir is None:
338             cache_dir = default_cache_dir(out_csv)
339         os.makedirs(cache_dir, exist_ok=True)
340         det_jsonl, _ = _cache_paths(cache_dir)
341         cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
342         manifest = None if fresh else read_manifest(cache_dir)
343         resume = manifest_compatible(
344             manifest or {}, frames_dir=cache_key, weights=weights, sport=sport,
345             frames_in=frames_in, frames_total=len(frames),
346         )
347         if resume:
348             windows_done, det_by_fid = load_and_clean_cache(cache_dir)
349             logger.info(f"resuming from cache: {windows_done}/{windows_total} windows
already detected")
350         else:
351             if manifest is not None:
352                 logger.info("cache incompatible with this run -> starting fresh")
353             reset_cache(cache_dir)
354             windows_done, det_by_fid = 0, defaultdict(list)
355
356         base_manifest = {
357             "version": CACHE_VERSION,
358             "frames_dir": osp.abspath(frames_dir),
359             "weights": weights,
360             "sport": sport,
361             "frames_in": frames_in,
362             "frames_total": len(frames),
363             "windows_total": windows_total,
364             "windows_done": windows_done,
365         }
366         write_manifest(cache_dir, base_manifest)
367
368         # ---- detector pass over the un-cached windows ----------------------- #
369         remaining = samples[windows_done:]
370         if remaining:
371             _, transform_test = build_img_transforms(cfg)
372             try:
```

```
373                 _, seq_transform_test = build_seq_transforms(cfg)
374             except Exception:
375                 seq_transform_test = None
376             ds = ImageDataset(cfg, remaining, input_wh, output_wh,
377                               transform=transform_test, seq_transform=seq_transform_test,
is_train=False)
378             loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
num_workers=num_workers)
379             detector = build_detector(cfg)
380
381             t0 = time.time()
382             done = windows_done
383             win_idx = windows_done
384             log_every = max(1, log_every_batches)
385             flush_n = max(1, flush_every)
386             pending = []
387             logger.info(f"detecting on {len(remaining)} remaining windows "
388                         f"(total {windows_total}, batch={batch_size},
workers={num_workers})...")
389             det_f = open(det_jsonl, "a")
390             try:
391                 with torch.no_grad():
392                     for bi, batch in enumerate(loader):
393                         imgs, _hms, trans = batch[0], batch[1], batch[2]
394                         img_paths = [list(t) for t in batch[-1]]   # [frame_in][batch] ->
path
395                         batch_results, _ = detector.run_tensor(imgs, trans)
396                         nb = imgs.shape[0]
397                         for ib in range(nb):
398                             frames_payload = []
399                             for ie in sorted(batch_results[ib].keys()):
400                                 p = img_paths[ie][ib]
401                                 fid = _frame_id(p)
402                                 plain = [_det_plain(d) for d in batch_results[ib][ie]]
403                                 frames_payload.append([fid, plain])
404                                 det_by_fid[fid].extend(plain)
405                             pending.append({"w": win_idx, "f": frames_payload})
406                             win_idx += 1
407                         done += nb
408                         if len(pending) >= flush_n or done >= windows_total:
409                             _append_windows(det_f, pending)
410                             pending = []
411                             base_manifest["windows_done"] = done
412                             write_manifest(cache_dir, base_manifest)
413                         if (bi + 1) % log_every == 0 or done >= windows_total:
414                             el = time.time() - t0
415                             new_done = done - windows_done
416                             rate = new_done / el if el > 0 else 0.0
417                             eta = (windows_total - done) / rate if rate > 0 else 0.0
418                             logger.info(f"{done}/{windows_total} "
419                                         f"({100 * done // max(1, windows_total)}%) |
{rate:.1f} win/s | "
```

25. assistant (2026-06-15T16:38:12.489Z)

Now let me update the manifest `frames_dir` field to use `cache_key`, and wrap the DataLoader iteration with the streaming `cv2.imread` shim. First the manifest:

26. user (2026-06-15T16:38:14.286Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors\wasb_infer.py has been updated successfully. (file state is current in your context ? no need to Read it back)

27. assistant (2026-06-15T16:38:20.986Z)

Now wire in the streaming shim. I need to:
1. Force `num_workers=0` in streaming mode.
2. Set up the `cv2.imread` shim that maps synthetic path ? decoded frame via `SequentialFrameStore`, scoped to the DataLoader iteration.

But there's a subtlety with the resumable cache: when resuming, the detector only processes `remaining = samples[windows_done:]`. The streaming reader must skip ahead to the first frame of `remaining`. The `SequentialFrameStore` is forward-only and starts at index 0. When resuming at window `windows_done`, the first frame needed is index `windows_done` (window `i` starts at frame `i`). The `frame_loader` (cv2.VideoCapture-based) must therefore seek/decode forward to that index. I'll handle this by having the streaming reader decode-and-discard up to the first needed index ? or use `CAP_PROP_POS_FRAMES`. Let me make `frame_loader` itself handle forward-only sequential reads, and have the store request the starting index first.

Let me update the detector-pass section:

28. user (2026-06-15T16:38:31.844Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors\wasb_infer.py has been updated successfully. (file state is current in your context ? no need to Read it back)

29. assistant (2026-06-15T16:38:37.361Z)

The added `with stream_ctx:` introduces an extra indentation level for the whole body. Using a 2-space indent for `with stream_ctx:` keeps the inner block at its original indentation ? that's a valid but unusual style. Let me reconsider: mixing indentation widths is fragile and ruff/flake may complain. A cleaner approach is to wrap the entire detector loop in a helper, or use `ExitStack`.

Actually, the cleanest is to not nest ? combine both context managers. Let me revert that hack and instead enter the streaming context as part of the existing `try`. Let me look at the full block again and restructure properly.

30. user (2026-06-15T16:38:38.682Z)

```
368         # ---- detector pass over the un-cached windows ----------------------- #
369         remaining = samples[windows_done:]
370         if remaining:
371             _, transform_test = build_img_transforms(cfg)
372             try:
373                 _, seq_transform_test = build_seq_transforms(cfg)
374             except Exception:
375                 seq_transform_test = None
376             ds = ImageDataset(cfg, remaining, input_wh, output_wh,
377                               transform=transform_test, seq_transform=seq_transform_test,
is_train=False)
378             # Streaming forces single-process loading: the in-memory frame store + the
379             # cv2.imread shim live in THIS process and can't be pickled to DataLoader
workers.
380             loader_workers = 0 if streaming else num_workers
381             loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
num_workers=loader_workers)
382             detector = build_detector(cfg)
383
384             # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)` to the
385             # in-memory decoded frame for that synthetic path. The detector's first needed
386             # frame is `windows_done` (window i starts at frame i), so prime the store
there.
387             stream_ctx = (_streaming_imread_patch(frame_loader, len(frames),
start_index=windows_done)
388                           if streaming else _nullcontext())
```

```
389
390             t0 = time.time()
391             done = windows_done
392             win_idx = windows_done
393             log_every = max(1, log_every_batches)
394             flush_n = max(1, flush_every)
395             pending = []
396             logger.info(f"detecting on {len(remaining)} remaining windows "
397                         f"(total {windows_total}, batch={batch_size},
workers={loader_workers}, "
398                         f"source={'video-stream' if streaming else 'frames-dir'})...")
399             det_f = open(det_jsonl, "a")
400             try:
401               with stream_ctx:
402                 with torch.no_grad():
403                     for bi, batch in enumerate(loader):
404                         imgs, _hms, trans = batch[0], batch[1], batch[2]
405                         img_paths = [list(t) for t in batch[-1]]   # [frame_in][batch] ->
path
406                         batch_results, _ = detector.run_tensor(imgs, trans)
407                         nb = imgs.shape[0]
408                         for ib in range(nb):
409                             frames_payload = []
410                             for ie in sorted(batch_results[ib].keys()):
411                                 p = img_paths[ie][ib]
412                                 fid = _frame_id(p)
413                                 plain = [_det_plain(d) for d in batch_results[ib][ie]]
414                                 frames_payload.append([fid, plain])
415                                 det_by_fid[fid].extend(plain)
416                             pending.append({"w": win_idx, "f": frames_payload})
417                             win_idx += 1
418                         done += nb
419                         if len(pending) >= flush_n or done >= windows_total:
420                             _append_windows(det_f, pending)
421                             pending = []
422                             base_manifest["windows_done"] = done
423                             write_manifest(cache_dir, base_manifest)
424                         if (bi + 1) % log_every == 0 or done >= windows_total:
425                             el = time.time() - t0
426                             new_done = done - windows_done
427                             rate = new_done / el if el > 0 else 0.0
428                             eta = (windows_total - done) / rate if rate > 0 else 0.0
429                             logger.info(f"{done}/{windows_total} "
430                                         f"({100 * done // max(1, windows_total)}%) |
{rate:.1f} win/s | "
431                                         f"elapsed {el:.0f}s | ETA {eta:.0f}s")
432                     if pending:
433                         _append_windows(det_f, pending)
434                         base_manifest["windows_done"] = done
435                         write_manifest(cache_dir, base_manifest)
436             finally:
437                 det_f.close()
438             logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
439         else:
440             logger.info("detector cache already complete -> tracker-only re-run")
441
442         # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
```

31. assistant (2026-06-15T16:38:42.482Z)

The 2-space `with stream_ctx:` indent works (the inner `with torch.no_grad()` stays at 16 spaces, which is now a nested block under the 10-space `with`). But this mixed indentation is ugly and ruff's E111 could flag it. Let me use `contextlib.ExitStack` to combine both contexts into a single clean `with`. Let me rewrite this block cleanly.

32. user (2026-06-15T16:38:50.235Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors\wasb_infer.py has been updated successfully. (file state is current in your context ? no need to Read it back)

33. assistant (2026-06-15T16:38:53.962Z)

Now I removed the `_nullcontext` reference, so I don't need that helper. Now let me add the `_streaming_imread_patch` context manager. It needs to be a module-level helper. Let me add it near the other helpers, before `run()`. It will create a `SequentialFrameStore` and monkeypatch `cv2.imread` to resolve synthetic paths to in-memory frames.

34. user (2026-06-15T16:39:02.246Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors\wasb_infer.py has been updated successfully. (file state is current in your context ? no need to Read it back)

35. assistant (2026-06-15T16:39:04.638Z)

Now I need to import `contextmanager` at the top. Let me update the imports.

36. user (2026-06-15T16:39:06.613Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors\wasb_infer.py has been updated successfully. (file state is current in your context ? no need to Read it back)

37. assistant (2026-06-15T16:39:11.229Z)

Now I need to add the streaming video decode driver and wire it into `main()` / `extract_frames`. The streaming path needs:
1. A function that opens a `cv2.VideoCapture` and produces a `frame_loader(index)` (forward-only) plus the total frame count, plus the ordered synthetic frame-path list.
2. `main()` to use it when `--stream-video` is passed.

Let me add a `stream_video_frames` helper and a `run_video_streaming` orchestration that builds the loader and calls `run(...)`. Let me view the `extract_frames` and `main` section first.

38. user (2026-06-15T16:39:12.258Z)

```
490         for fid in sorted(det_by_fid.keys()):
491             r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
492             rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))
493 
494         _atomic_write_trajectory(out_csv, rows)
495         visible = sum(1 for _, v, *_ in rows if v)
496         logger.info(f"wrote {len(rows)} rows -> {out_csv}")
497         logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
498         return out_csv
499 
500 
501     def extract_frames(video_path: str, frames_dir: str) -> str:
502         """Decode a video to PNG frames named by source frame index (00000000.png ...).
503 
504         Idempotent: if the dir already holds the expected number of frames (matching the
505         container's reported frame count) we skip re-decoding ? re-extraction after a
506         reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
507         extracting with ffmpeg on the host and passing --frames_dir.
508         """
509         import cv2
510         os.makedirs(frames_dir, exist_ok=True)
511         cap = cv2.VideoCapture(video_path)
```

```
512         if not cap.isOpened():
513             raise SystemExit(f"cannot open video: {video_path}")
514         expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
515         existing = len(glob.glob(osp.join(frames_dir, "*.png")))
516         if expected > 0 and existing >= expected:
517             cap.release()
518             logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
519             return frames_dir
520         i = 0
521         while True:
522             ok, frame = cap.read()
523             if not ok:
524                 break
525             cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
526             i += 1
527         cap.release()
528         logger.info(f"extracted {i} frames -> {frames_dir}")
529         return frames_dir
530
531
532     def main():
533         ap = argparse.ArgumentParser()
534         ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
535         ap.add_argument("--video", help="video file; frames are extracted internally (in
WSL)")
536         ap.add_argument("--frames_out_dir", help="where to extract frames when --video is
used")
537         ap.add_argument("--weights", required=True)
538         ap.add_argument("--out", required=True)
539         ap.add_argument("--sport", default="badminton")
540         ap.add_argument("--limit", type=int, default=0, help="cap #frames (for quick
tests)")
541         ap.add_argument("--batch-size", type=int, default=8)
542         ap.add_argument("--num-workers", type=int, default=4)
543         ap.add_argument("--log-every-batches", type=int, default=50, help="progress log
cadence")
544         ap.add_argument("--cache-dir", default=None,
545                         help="resumable detector cache dir (default: <out>_wasbcache next to
--out)")
546         ap.add_argument("--flush-every", type=int, default=1000,
547                         help="fsync the detector cache every N windows (bounds reboot
loss)")
548         ap.add_argument("--fresh", action="store_true", help="ignore any existing cache and
start over")
549         args = ap.parse_args()
550
551         frames_dir = args.frames_dir
552         if args.video:
553             frames_dir = extract_frames(args.video, args.frames_out_dir or (args.video +
"_frames"))
554         if not frames_dir:
555             raise SystemExit("provide --frames_dir or --video")
556         run(frames_dir, args.weights, args.out, args.sport, args.limit,
557             batch_size=args.batch_size, num_workers=args.num_workers,
558             log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
559             flush_every=args.flush_every, fresh=args.fresh)
560
561
562     if __name__ == "__main__":
563         main()
564
```

39. assistant (2026-06-15T16:39:15.633Z)

Now let me add the streaming video helpers and wire them into `main()`. I'll add a

`_video_frame_count`, a `make_video_frame_loader` (forward-only sequential decoder with strict ordering + a guarded random-access seek fallback for resume), and a `run_video_streaming` orchestration. Let me insert these after `extract_frames`.

40. user (2026-06-15T16:39:41.934Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors\wasb_infer.py has been updated successfully. (file state is current in your context ? no need to Read it back)

41. assistant (2026-06-15T16:39:50.913Z)

Now there's a subtlety with the streaming resume + `SequentialFrameStore`. When resuming, `_streaming_imread_patch` primes the store at `start_index=windows_done` via `store.get(windows_done)`. That calls `frame_loader` for indices `0..windows_done`, which would read from index 0 forward ? but we want to *skip* to `windows_done` without decoding `0..windows_done-1`.

The `SequentialFrameStore.get(start_index)` pulls forward from `self._next=0`, calling `reader(0), reader(1), ..., reader(start_index)` ? decoding every frame up to the resume point. That defeats the resume speedup (and is slow). I should instead make the store seekable for the initial prime: set `_next` and `_low` to `start_index` and let the `frame_loader` itself seek.

Let me reconsider. The cleanest: the store should support being told "start at index N" so it doesn't decode 0..N-1. The `frame_loader` already handles forward-skip via `cap.set(POS_FRAMES, index)`. So the store just needs to begin pulling at `start_index`. Let me add a `start_index` to `SequentialFrameStore.__init__` that sets `_next` and `_low` to it, and have `_streaming_imread_patch` pass it through instead of calling `get()` to prime.

42. user (2026-06-15T16:39:55.274Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors\wasb_infer.py has been updated successfully. (file state is current in your context ? no need to Read it back)

43. user (2026-06-15T16:39:59.973Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors\wasb_infer.py has been updated successfully. (file state is current in your context ? no need to Read it back)

44. assistant (2026-06-15T16:40:05.050Z)

Now there's a `manifest_compatible` parameter naming issue. The function signature uses `frames_dir=` as the keyword. I'm passing `frames_dir=cache_key`. That's fine ? the manifest stores it under the `frames_dir` key. But for the streaming path, the `frames_dir` field in the manifest now holds `"video:<abspath>"`. That's consistent between write and compatibility-check. Good.

Now let me wire the runner (`wasb_runner.py`) to use the streaming path, gated by config. Let me add a `stream_video` config field (default `False` so production behavior is unchanged) and pass `--stream-video` when enabled. When streaming, no `--frames_out_dir` is needed and there's no frame cache to clean up.

45. user (2026-06-15T16:40:16.602Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors\wasb_runner.py has been updated successfully. (file state is current in your context ? no need to Read it back)

46. assistant (2026-06-15T16:40:20.655Z)

Now update `run_predict` to pass `--stream-video` when configured, and skip frame-cache cleanup in that mode. Let me update the command construction and the cleanup.

47. user (2026-06-15T16:40:31.107Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors\wasb_runner.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

48. assistant (2026-06-15T16:40:34.552Z)

Now the cleanup at the end removes `frames_dir` + `staged_video`. In streaming mode, `frames_dir` is never created, so removing it is harmless (rm -rf of a nonexistent path is a no-op). But to keep things clean, let me only clean up the frames dir when not streaming.

49. user (2026-06-15T16:40:35.500Z)

```
176             if res.returncode != 0:
177                 logger.error("WASB inference failed:\n%s", (res.stderr or
res.stdout)[-2000:])
178                 return None
179             if not os.path.exists(expected_csv_win):
180                 logger.error("WASB finished but expected CSV not found at %s",
expected_csv_win)
181                 return None
182             logger.info("WASB trajectory CSV: %s", expected_csv_win)
183
184             # Success ? the trajectory CSV is the durable output; the staged video copy and
185             # the (huge) decoded frame cache are now pure intermediates. Delete them unless
186             # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
187             # so the frame cache + detector cache remain for a resumable re-run.
188             if not self.cfg.keep_frames:
189                 logger.info("Cache hygiene: removing staged video + frame cache for %s "
190                             "(set indexing.wasb.keep_frames=true to retain).", key)
191                 self._wsl_rm_rf([frames_dir, staged_video])
192             return expected_csv_win
193
194         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
195         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
196         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
197
```

50. user (2026-06-15T16:40:43.341Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors\wasb_runner.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

51. assistant (2026-06-15T16:40:46.355Z)

Now let me verify the full edited `wasb_infer.py` reads cleanly around the key sections, then run the linters and tests. Let me re-read the `run` signature area and the streaming patch to ensure consistency.

52. user (2026-06-15T16:40:47.888Z)

```
205     # ---------------------------------------------------------------------------- #
206     # Frame addressing + windowing (pure; no torch/cv2 ? unit-testable)
207     # ---------------------------------------------------------------------------- #
208     _STREAM_FRAME_FMT = "{:08d}.png"
209
210
211     def synthetic_frame_path(index: int) -> str:
212         """Stable, index-encoding frame name used by the streaming path.
213
```

```
        It mirrors the on-disk names ``extract_frames`` writes (``{i:08d}.png``) so the
        downstream integer-id parser (``_frame_id``) yields the SAME frame id whether the
        frames came from disk or from the in-memory stream. The streaming image loader
        inverts this name back to an index to fetch the decoded frame.
        """
        return _STREAM_FRAME_FMT.format(int(index))


    def build_windows(frames: list, frames_in: int):
        """Sliding windows of ``frames_in`` consecutive frame *paths* (the unit of GPU
        work + checkpoint). Pure list math, identical for the disk and streaming paths.

        Returns a list of ``{"frames": [paths], "annos": [{frame_path, center}]}`` where
        ``center`` is the WASB ``Center`` placeholder (filled lazily by the caller so this
        stays import-light / torch-free for unit tests).
        """
        return [frames[i:i + frames_in] for i in range(len(frames) - frames_in + 1)]


    class SequentialFrameStore:
        """In-memory sliding buffer over a forward-only frame reader, sized for the
        detector's strictly-increasing, overlap-by-(frames_in-1) access pattern.

        The detector DataLoader runs with ``shuffle=False`` and (in streaming mode)
        ``num_workers=0``, so frames are requested in non-decreasing index order; each
        sliding window re-reads at most ``frames_in - 1`` already-seen frames. We hold a
        small dict buffer and evict any index below the lowest one still in flight, so
        peak memory is O(window span) instead of O(whole video). ``reader(index)`` must
        return the decoded frame for ``index`` (forward-only); it is called exactly once
        per source frame, in order.
        """

        def __init__(self, reader, total: int, start_index: int = 0):
            self._reader = reader
            self._total = int(total)
            self._buf: dict = {}
            # On a resumed run the detector's first needed frame is `start_index`; begin
            # pulling there so the reader can skip (seek past) the already-cached prefix
            # instead of decoding 0..start_index-1 just to throw them away.
            self._next = int(start_index)   # next index not yet pulled from the reader
            self._low = int(start_index)    # lowest index we still keep (eviction floor)

        def get(self, index: int):
            index = int(index)
            if index < self._low:
                raise KeyError(
                    f"frame {index} already evicted (below floor {self._low}); the streaming
"
                    f"reader is forward-only and access must be non-decreasing")
            if index >= self._total:
                raise KeyError(f"frame {index} out of range (total {self._total})")
            # Pull forward from the reader until the requested index is buffered.
            while self._next <= index:
                self._buf[self._next] = self._reader(self._next)
                self._next += 1
            # Evict everything strictly below the requested index: windows only ever look
            # back within the current window, never before the frame just asked for.
            if index > self._low:
                for k in range(self._low, index):
                    self._buf.pop(k, None)
                self._low = index
            return self._buf[index]


    def _index_from_synthetic_path(path: str) -> int:
```

```
278         """Invert ``synthetic_frame_path``: a frame path -> its integer source index.
279 
280         Reuses ``_frame_id`` (digits-only parse) so the index and the cached frame-id stay
281         in lockstep with the on-disk naming scheme.
282         """
283         return _frame_id(path)
284 
285 
286     @contextmanager
287     def _streaming_imread_patch(frame_loader, total: int, start_index: int = 0):
288         """Scope a ``cv2.imread`` shim that returns in-memory decoded frames during the
289         detector pass, so ``ImageDataset`` runs byte-for-byte as it would over real PNGs.
290 
291         ``frame_loader(index) -> BGR uint8 ndarray`` is a forward-only sequential decoder;
292         we wrap it in a ``SequentialFrameStore`` (small sliding buffer). On a resumed run we
293         prime the store at ``start_index`` so the decoder skips already-cached windows. The
294         original ``cv2.imread`` is restored on exit. Any path the shim can't map to a known
295         frame index falls through to the real ``cv2.imread`` (defensive ? shouldn't happen).
296         """
297         import cv2
298         store = SequentialFrameStore(frame_loader, total, start_index=start_index)
299         real_imread = cv2.imread
300 
301         def _shim(path, *args, **kwargs):
302             idx = _index_from_synthetic_path(path)
303             if idx < 0:
304                 return real_imread(path, *args, **kwargs)
305             return store.get(idx)
306 
307         cv2.imread = _shim
308         try:
309             yield store
310         finally:
311             cv2.imread = real_imread
312 
313 
314     # ---------------------------------------------------------------------- #
315     # Inference
316     # ---------------------------------------------------------------------- #
317     def run(frames, weights: str, out_csv: str, sport: str = "badminton",
318             limit: int = 0, batch_size: int = 8, num_workers: int = 4,
319             log_every_batches: int = 50, cache_dir: "str | None" = None,
320             flush_every: int = 1000, fresh: bool = False,
321             frame_loader=None, source_key: "str | None" = None) -> str:
322         """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
323 
324         Two equivalent ways to supply pixels (output is bit-identical between them):
```

53. assistant (2026-06-15T16:40:51.461Z)

There's a subtle but important bug in the build_windows docstring ? it says it returns dicts but actually returns lists of paths. Let me fix the docstring to match the actual return (lists of path-windows; the caller builds the dict). The current docstring is misleading.

54. user (2026-06-15T16:40:57.984Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors\wasb_infer.py has been updated successfully. (file state is current in your context ? no need to Read it back)

55. assistant (2026-06-15T16:41:00.842Z)

Now let me run ruff and mypy on the changed files, and check for import-time issues. Let me check which tools are available.

56. user (2026-06-15T16:41:05.032Z)

```
ruff 0.15.16
---mypy---
mypy 2.1.0 (compiled: yes)
---pytest---
pytest 9.0.3
```

57. assistant (2026-06-15T16:41:07.971Z)

All tools available. Let me run ruff and mypy on the two changed files, plus check the CI config for the exact mypy invocation.

58. user (2026-06-15T16:41:10.423Z)

```
All checks passed!
```

59. user (2026-06-15T16:41:10.810Z)

```
ci.yml
```

60. user (2026-06-15T16:41:15.142Z)

```
14-
15-jobs:
16:  # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
17:  # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
18-  # tracked .py, including scripts/ and training/, which the backend-only
19-  # py_compile smoke sweep does not reach.
--
26-        with:
27-          python-version: "3.13"
28:      - name: Install ruff
29:        run: python -m pip install ruff
30:      - name: Ruff check
31:        run: ruff check . --output-format github
32-
33:  # Type gate (mypy.ini at repo root): lenient green baseline with an explicit
34-  # per-module quarantine list ? ratchet by removing entries as modules clean
35:  # up. Typed core deps are installed so mypy sees their real signatures
36-  # (without them ignore_missing_imports would Any-stub half the checks).
37-  typecheck:
--
43-        with:
44-          python-version: "3.13"
45:      - name: Install mypy + typed core deps
46:        run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-genai
numpy
47:      - name: Mypy
48:        run: mypy backend training
49-
50-  # Fast guard against the 7fbbeb0 class of breakage: a SyntaxError or
--
85:      - name: Install package (editable) + dev tooling
86:        # Editable PEP 621 install: runtime deps + the `dev` group
87:        # (pytest, pytest-cov, httpx for the fastapi TestClient, ruff, mypy).
88:        run: python -m pip install -e .[dev]
89-      # obsreport (private, soft dep) is absent here: reporting tests skip via
```

61. assistant (2026-06-15T16:41:18.330Z)

CI runs `mypy backend training`. Let me check the mypy.ini quarantine list to see if wasb_infer/wasb_runner are quarantined, then run mypy.

62. user (2026-06-15T16:42:32.496Z)

```
backend\pipeline\detectors\wasb_infer.py:308: error: Incompatible types in assignment
(expression has type "def _shim(path: Any, *args: Any, **kwargs: Any) -> Any", variable has type
overloaded function)  [assignment]
Found 1 error in 1 file (checked 2 source files)
```

63. assistant (2026-06-15T16:42:37.042Z)

mypy flags the `cv2.imread` reassignment (cv2 has an overloaded stub). Let me check how the quarantine list and `mypy.ini` handle this ? but cleaner to fix it directly. I'll assign via `setattr` to avoid the overload-typed attribute assignment error. Let me check mypy.ini first to understand the baseline.

64. user (2026-06-15T16:42:38.243Z)

```
[mypy]
```

Lenient, GREEN baseline (quality sweep AI-4): the gate protects the clean modules and pins an explicit quarantine list below ? not a typing crusade. Ratchet: when you touch a quarantined module, try removing its entry.

```
ignore_missing_imports = True
```

backend/ is a namespace package (no root __init__.py) ? required so mypy maps files to backend.* module names instead of duplicating top-level names.

```
explicit_package_bases = True
```

--- Quarantine (known-loose modules, each with a reason) ---

Optional module globals (db_instance/global_config set only in main()); properly fixed by the deferred app-factory/lifespan refactor (BACKLOG).

```
[mypy-backend.main]
ignore_errors = True
```

Legacy demo segmenter (fabricated metadata; coverage-omitted too).

```
[mypy-backend.pipeline.segmenters.motion]
ignore_errors = True
```

yolo_hybrid: the fusion-P1 extraction moved its loose torchvision/cv2 inline code to detectors/person_detector.py (which IS type-clean, no quarantine). What remains here is pre-existing orchestration debt unrelated to that move (implicit-Optional process_video defaults, the Success|Failure ABC-override mismatch, a Future[...] annotation) ? ratchet those out in a focused typing pass, not here. (wasb_hybrid/tracknet_hybrid left the quarantine with the AI-7 collapse.)

```
[mypy-backend.pipeline.segmenters.yolo_hybrid]
ignore_errors = True

[mypy-backend.pipeline.segmenters.gemini]
ignore_errors = True
```

Registry builds cls(None, {}) probes; typing it honestly needs a Protocol.

```
[mypy-backend.pipeline.segmenters.base]
ignore_errors = True
```

65. assistant (2026-06-15T16:42:42.540Z)

`wasb_infer` is NOT quarantined, so it must stay mypy-clean. Let me fix the `cv2.imread` reassignment using `setattr`/`getattr`, which sidesteps the overloaded-attribute assignment error cleanly.

66. user (2026-06-15T16:42:49.749Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors\wasb_infer.py has been updated successfully. (file state is current in your context ? no need to Read it back)

67. user (2026-06-15T16:43:08.694Z)

```
Exit code 1
Success: no issues found in 2 source files
===ruff===
B010 [*] Do not call `setattr` with a constant attribute value. It is not any safer than normal
property access.
   --> backend\pipeline\detectors\wasb_infer.py:310:5
    |
308 |     # setattr/restore (not `cv2.imread = ...`) so mypy doesn't object to rebinding the
309 |     # overloaded stub; the shim is in scope only for the detector pass.
310 |     setattr(cv2, "imread", _shim)
    |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
311 |     try:
312 |         yield store
    |
help: Replace `setattr` with assignment

B010 [*] Do not call `setattr` with a constant attribute value. It is not any safer than normal
property access.
   --> backend\pipeline\detectors\wasb_infer.py:314:9
    |
312 |         yield store
313 |     finally:
314 |         setattr(cv2, "imread", real_imread)
    |         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    |
help: Replace `setattr` with assignment

Found 2 errors.
[*] 2 fixable with the `--fix` option.
```

68. assistant (2026-06-15T16:43:12.225Z)

mypy passes now but ruff's bugbear B010 dislikes `setattr` with a constant. The two linters conflict. The cleanest resolution: keep the direct assignment `cv2.imread = _shim` and add a targeted `# type: ignore[assignment]` for mypy. Let me do that.

69. user (2026-06-15T16:43:15.671Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors\wasb_infer.py has been updated successfully. (file state is current in your context ? no need to Read it back)

70. user (2026-06-15T16:43:22.703Z)

```
backend\pipeline\detectors\wasb_infer.py:309: error: Invalid "type: ignore" comment  [syntax]
Found 1 error in 1 file (checked 2 source files)
===ruff===
All checks passed!
```

71. assistant (2026-06-15T16:43:26.671Z)

Ruff is happy. mypy complains about an "invalid type: ignore comment" at line 309 ? that's

because the comment with text after `# type: ignore[assignment]` on the same logical area...
Actually the error is on line 309, which is the comment line, not the assignment. Let me look ?
the multi-line comment block might be confusing mypy. Let me check the exact lines.

72. user (2026-06-15T16:43:27.622Z)

```
300         real_imread = cv2.imread
301
302         def _shim(path, *args, **kwargs):
303             idx = _index_from_synthetic_path(path)
304             if idx < 0:
305                 return real_imread(path, *args, **kwargs)
306             return store.get(idx)
307
308         # Rebind cv2.imread for the duration of the detector pass; restore on exit. The
309         # type: ignore silences mypy rebinding the overloaded stub (ruff's B010 forbids
310         # setattr-with-constant, so plain assignment is the lint-clean form).
311         cv2.imread = _shim  # type: ignore[assignment]
312         try:
313             yield store
314         finally:
315             cv2.imread = real_imread  # type: ignore[assignment]
```

73. assistant (2026-06-15T16:43:31.481Z)

The literal text "type: ignore" inside my comment on line 309 is being parsed by mypy as a malformed `# type: ignore` directive. I'll reword the comment to avoid the phrase "type: ignore".

74. user (2026-06-15T16:43:34.944Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors\wasb_infer.py has been updated successfully. (file state is current in your context ? no need to Read it back)

75. user (2026-06-15T16:43:41.726Z)

```
Success: no issues found in 2 source files
===ruff===
All checks passed!
```

76. assistant (2026-06-15T16:43:44.070Z)

Both clean. Now let me run the existing tests for the touched modules and the import smoke test.

77. user (2026-06-15T16:44:01.765Z)

```
....................................                                     [100%]
40 passed in 15.01s
```

78. assistant (2026-06-15T16:44:07.493Z)

All existing tests pass. Now let me add unit tests for the new pure helpers and the streaming wiring. I'll add them to `test_wasb_cache.py` (pure wasb_infer pieces live there) and add a runner-wiring test to `test_tracknet_runner.py`. Let me first verify the streaming path works end-to-end logically with a fake by writing a focused test. Let me add tests to `test_wasb_cache.py`.

79. user (2026-06-15T16:44:08.543Z)

```
152
153     # ---------------------------------------------------------------------- #
154     # misc helpers
```

```
155     # ---------------------------------------------------------------------------- #
156     def test_default_cache_dir_is_next_to_output():
157         out = osp.join("some", "dir", "sample_long_traj.csv")
158         cd = wi.default_cache_dir(out)
159         assert osp.basename(cd) == "sample_long_traj_wasbcache"
160         assert osp.dirname(cd) == osp.dirname(osp.abspath(out))
161 
162 
163     def test_atomic_write_trajectory(tmp_path):
164         out = str(tmp_path / "out" / "traj.csv")
165         wi._atomic_write_trajectory(out, [(0, 1, 439.0, 64.0), (1, 0, -1, -1)])
166         assert not osp.exists(out + ".tmp")
167         with open(out) as f:
168             text = f.read().splitlines()
169         assert text[0] == "Frame,Visibility,X,Y"
170         assert text[1] == "0,1,439.0,64.0"
171         assert text[2] == "1,0,-1,-1"
172 
```

80. assistant (2026-06-15T16:44:14.159Z)

Now let me add a new test section for the streaming-path pure helpers: `synthetic_frame_path`/`_index_from_synthetic_path` round-trip, `build_windows`, `SequentialFrameStore` (forward-only access, eviction, resume start_index, out-of-range/backward errors), and the `_streaming_imread_patch` shim resolving synthetic paths and restoring cv2. The shim test needs cv2; let me make it skip gracefully if cv2 isn't importable so the test stays collectible everywhere. Let me check whether cv2 is importable here.

81. user (2026-06-15T16:44:16.166Z)

```
cv2 4.13.0
numpy 2.4.6
```

82. assistant (2026-06-15T16:44:20.285Z)

cv2 and numpy are both available here. But the task says tests must be "mockable/pure (no torch/cv2/GPU needed at collection)". To be safe and portable (CI may not have cv2), I'll keep the pure-logic tests free of cv2, and make the one shim test that touches `cv2.imread` import cv2 lazily inside the test with a `pytest.importorskip` guard. Let me write the tests.

83. user (2026-06-15T16:44:26.048Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\agent-a771ff1d85e18f2ed\tests\test_wasb_cache.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

84. user (2026-06-15T16:44:52.642Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\agent-a771ff1d85e18f2ed\tests\test_wasb_cache.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

85. assistant (2026-06-15T16:44:56.125Z)

The "fall through to real imread" assertion is sloppy (`is None or True` always passes). Let me make it a meaningful assertion: when a path has no parseable index, the shim calls real imread, which returns `None` for a nonexistent file. Let me tighten it.

86. user (2026-06-15T16:45:04.065Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\agent-a771ff1d85e18f2ed\tests\test_wasb_cache.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

87. assistant (2026-06-15T16:45:08.329Z)

Now let me add a runner-wiring test to `test_tracknet_runner.py` to verify: (1) config parses `stream_video`, (2) `run_predict` passes `--stream-video` and NOT `--frames_out_dir` when streaming, and (3) the default path still passes `--frames_out_dir`. I'll capture the command via patching `_wsl_bash`.

88. user (2026-06-15T16:45:21.215Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\agent-a771ff1d85e18f2ed\tests\test_tracknet_runner.py has been updated successfully. (file state is current in your context ? no need to Read it back)

89. assistant (2026-06-15T16:45:25.141Z)

Now let me run the new and existing tests for the touched modules, plus ruff/mypy on the test files (mypy CI only checks `backend training`, not tests, but ruff checks everything).

90. user (2026-06-15T16:45:29.829Z)

```
================================== FAILURES ===================================
________ test_store_reads_each_frame_once_in_order_for_sliding_windows ________

    def test_store_reads_each_frame_once_in_order_for_sliding_windows():
        # Mimic the detector's access pattern: windows (0,1,2),(1,2,3),(2,3,4) over 5 frames
        # request frame indices 0,1,2, 1,2,3, 2,3,4 ? every source frame decoded exactly once.
        reader, calls = _counting_reader()
        store = wi.SequentialFrameStore(reader, total=5)
        order = [0, 1, 2, 1, 2, 3, 2, 3, 4]
>       got = [store.get(i) for i in order]
               ^^^^^^^^^^^^

tests\test_wasb_cache.py:227:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

self = <backend.pipeline.detectors.wasb_infer.SequentialFrameStore object at 0x000002D3FFA9AA50>
index = 1

    def get(self, index: int):
        index = int(index)
        if index < self._low:
>           raise KeyError(
                f"frame {index} already evicted (below floor {self._low}); the streaming "
                f"reader is forward-only and access must be non-decreasing")
E           KeyError: 'frame 1 already evicted (below floor 2); the streaming reader is forward-only and access must be non-decreasing'

backend\pipeline\detectors\wasb_infer.py:260: KeyError
=========================== short test summary info ===========================
FAILED tests/test_wasb_cache.py::test_store_reads_each_frame_once_in_order_for_sliding_windows
1 failed, 46 passed in 1.55s
```

91. assistant (2026-06-15T16:45:46.745Z)

This is a critical finding about my eviction logic. My test models the access pattern as `0,1,2, 1,2,3, 2,3,4` ? i.e. within each window the indices go up, but **between windows the index drops back** (window 1 starts at frame 1 after window 0 ended at frame 2). My eviction floor advances to the *max* index requested, so frame 1 gets evicted after frame 2 is requested, then window 1 asks for frame 1 again ? KeyError.

So my assumption "access is non-decreasing" is **wrong**. The real access pattern across windows is: window `i` requests frames `i, i+1, i+2`, then window `i+1` requests `i+1, i+2, i+3`. So the sequence is `0,1,2,1,2,3,2,3,4,...` ? it dips back by `frames_in-1` at each window boundary.

The eviction floor must therefore be the **lowest frame index still needed by any upcoming window** ? which is the *start* of the current window, not the max index seen. Since windows advance by 1 and are processed in order, after finishing window `i` the next window starts at `i+1`, so frame `i` can be evicted but frames `i+1, i+2` are still needed.

But the store doesn't know window boundaries ? it only sees individual `get(index)` calls. The safe eviction rule: evict frames below `(max_index_seen - (frames_in - 1))`. But the store doesn't know `frames_in` either.

The cleanest fix: the eviction floor should be based on the **minimum** index that could still be requested. Since DataLoader requests window samples in order and each `__getitem__` reads its window's frames left-to-right, the lowest index that can appear *after* the current call is bounded by the window structure. Without knowing `frames_in`, the simplest correct approach is to keep a **bounded LRU-style buffer of the last K frames** where K ? window span.

Actually the simplest, provably-correct fix: pass `frames_in` (the window span) to the store so it only evicts frames older than `current_max - (frames_in - 1)`. Let me reconsider the access order precisely.

For `__getitem__(sample_i)`, the dataset reads frames `[i, i+1, ..., i+frames_in-1]`. Samples are requested `i = start, start+1, ...`. So the global read sequence is:
- i=start: start, start+1, ..., start+W-1
- i=start+1: start+1, ..., start+W
- ...

where W = frames_in. The minimum index ever requested again after we're at sample `i` reading frame `f` is `i` (the start of the current window) ? but actually once we move to sample `i+1`, the minimum is `i+1`. The peak buffer needed is W frames. The lowest index still needed when processing sample `i` is `i`.

So eviction floor = the **start index of the lowest in-flight window**. Since the store sees reads `i, i+1, ..., i+W-1` then `i+1, ...`, the start-of-window is the value at which the index *resets downward*. I can track this: maintain `_low` = `max(0, max_index_seen - (W - 1))`. With W known, evicting everything below `max_seen - (W-1)` is always safe. Let me pass `window=frames_in` to the store and use that.

92. user (2026-06-15T16:46:03.484Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors\wasb_infer.py has been updated successfully. (file state is current in your context ? no need to Read it back)

93. assistant (2026-06-15T16:46:06.345Z)

Now I need to thread `window=frames_in` through `_streaming_imread_patch` and the `run()` call site. Let me update the patch signature and the call.

94. user (2026-06-15T16:46:13.892Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors\wasb_infer.py has been updated successfully. (file state is current in your context ? no need to Read it back)

95. user (2026-06-15T16:46:18.690Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors\wasb_infer.py has been updated successfully. (file state is current in your context ? no need to Read it back)

96. assistant (2026-06-15T16:46:21.877Z)

Now I need to update my tests to pass `window=3` (matching frames_in) where the sliding-window pattern is exercised, and the eviction test. Let me update the affected tests.

97. user (2026-06-15T16:46:38.587Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\agent-a771ff1d85e18f2ed\tests\test_wasb_cache.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

98. user (2026-06-15T16:47:15.062Z)

```
.................................................                    [100%]
53 passed in 31.71s
```

99. assistant (2026-06-15T16:47:18.777Z)

All 53 pass. Now let me verify there are no other callers of `wasb_infer.run(frames_dir=...)` that would break from the positional-arg rename (`frames_dir` ? `frames`), and check the calibrate/export drivers that import from these modules.

100. user (2026-06-15T16:47:21.271Z)

```
Found 40 files
tests\test_wasb_cache.py
backend\pipeline\detectors\wasb_infer.py
tests\test_tracknet_runner.py
backend\pipeline\detectors\wasb_runner.py
training\gen0\harness.py
tools\cleanup_caches.py
tests\test_wsl_tilde_quoting.py
tests\test_wasb_runner.py
tests\test_trajectory_hybrid_equivalence.py
tests\test_training_gen0.py
tests\test_tracknet_hybrid.py
tests\test_reliability_hardening.py
tests\test_rally_seq_proto.py
tests\test_import_smoke.py
tests\test_fusion_golden_telemetry.py
tests\test_fusion_hybrid.py
tests\test_fusion_compare.py
tests\test_export_wasb_segments.py
tests\test_eval_partial_labels.py
tests\test_config_wasb.py
tests\test_detector_keying.py
run_all_tests.py
backend\utils\video.py
backend\utils\run_telemetry.py
backend\tools\rally_slicer.py
backend\pipeline\validators\format.py
backend\pipeline\validators\pts_continuity.py
backend\pipeline\stitcher\compiler.py
backend\pipeline\segmenters\wasb_hybrid.py
backend\pipeline\segmenters\tracknet_hybrid.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\pipeline\segmenters\gemini.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\detectors\tracknet_runner.py
backend\pipeline\detectors\base.py
backend\pipeline\detectors\__init__.py
backend\pipeline\audio\hit_detector.py
backend\main.py
backend\eval\calibrate_wasb.py
backend\config\models.py
```

101. assistant (2026-06-15T16:47:24.781Z)

Let me check the direct callers of `wasb_infer.run` (to confirm none break from the signature

change) and `test_config_wasb.py` (config schema), and whether the config model needs a `stream_video` field for validation.

102. user (2026-06-15T16:47:26.354Z)

```
backend\pipeline\detectors\wasb_infer.py:596:def run_video_streaming(video_path: str, weights:
str, out_csv: str, sport: str = "badminton",
backend\pipeline\detectors\wasb_infer.py:647:        run_video_streaming(args.video,
args.weights, args.out, args.sport, args.limit,
tests\test_wasb_cache.py:19:from backend.pipeline.detectors import wasb_infer as wi
```

103. user (2026-06-15T16:47:27.780Z)

```
40-    stage_in_wsl: bool = True
41-    wsl_stage_dir: str = "~/clips"
42:    timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43-    velocity_threshold: float = 0.02
44-    # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
--
46-    # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47-    # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48:    keep_frames: bool = False
49-    # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50-    # 0 = disabled.
51:    min_free_gb: float = 0.0
52-
53-
--
66-    sport: str = "badminton"
67-    wsl_stage_dir: str = "~/clips"
68:    timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69-    velocity_threshold: float = 0.02
70-    # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
71-    # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates, regenerable
72-    # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
73:    keep_frames: bool = False
74-    # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
75:    min_free_gb: float = 0.0
76-
77-
```

104. assistant (2026-06-15T16:47:32.572Z)

Good ? `wasb_infer.run` has no external callers (only the module's own `main()`/`run_video_streaming`). The signature change is safe.

There's a Pydantic config model that mirrors the WASB indexing config. Since `WasbConfig.from_indexing_cfg` reads from a dict (`idx_cfg["wasb"]`), and the Pydantic model defines the validated schema, I should add `stream_video` to the Pydantic model so it's a recognized config field (otherwise it may be rejected as an extra field, depending on the model's `extra` policy). Let me inspect the WASB config model block.

105. user (2026-06-15T16:47:33.537Z)

```
1       """Pydantic models for the application configuration.
2
3       This module defines the canonical, validated, typed configuration for the
4       Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6       The models support graceful migration from old config.json files via extra="allow".
7       """
8
```

```
    from __future__ import annotations

    from typing import Any, List, Literal, Optional

    from pydantic import BaseModel, Field, field_validator, model_validator

    Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]


    class ValidationRelevanceConfig(BaseModel):
        court_green_lower: list[int] = Field(default=[35, 40, 40])
        court_green_upper: list[int] = Field(default=[85, 255, 255])
        court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)


    class ValidationConfig(BaseModel):
        min_resolution_width: int = 1280
        min_resolution_height: int = 720
        min_fps: float = 29.9
        min_blur_threshold: float = 20.0
        max_scene_cuts_per_minute: float = 0.5
        relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)


    class TrackNetConfig(BaseModel):
        wsl_distro: str = "Ubuntu"
        repo_dir: str = "~/models/TrackNetV4"
        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
        conda_env: str = "TrackNetV4"
        weights_path: str = ""
        queue_length: int = 5
        stage_in_wsl: bool = True
        wsl_stage_dir: str = "~/clips"
        timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
        velocity_threshold: float = 0.02
        # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
        # decoded frame cache) are deleted automatically ? they are pure intermediates,
        # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
        # true only for debugging. On FAILURE they are always kept so a re-run can resume.
        keep_frames: bool = False
        # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
        # 0 = disabled.
        min_free_gb: float = 0.0


    class WasbIndexConfig(BaseModel):
        """Typed config for the live WASB shuttle substrate (indexing.wasb).

        Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
        these knobs actually take effect ? previously this whole block was silently dropped
        because nested config sections defaulted to extra='ignore'.
        """
        wsl_distro: str = "Ubuntu"
        repo_dir: str = "~/models/WASB-SBDT"
        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
        conda_env: str = "wasb"
        weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
        sport: str = "badminton"
        wsl_stage_dir: str = "~/clips"
        timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
        velocity_threshold: float = 0.02
        # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
        # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
```

```
regenerable
72          # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
73          keep_frames: bool = False
74          # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
75          min_free_gb: float = 0.0
76
77
78      class FusionConfig(BaseModel):
79          """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
80
81          Every default reproduces control behaviour: the fusion segmenter persists the
82          shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
83          true ? the person-cue features, but it does NOT gate the served handoff unless a
84          threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
85          is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
86          supplied ? off-court persons (spectators / adjacent courts) are excluded.
87          """
88          # Which trajectory substrate seeds the windows (shuttle stays primary).
89          substrate: Literal["wasb", "tracknet"] = "wasb"
90          # Windowing velocity threshold for the fusion family (the engine reads
```

106. assistant (2026-06-15T16:47:38.242Z)

The docstring says nested sections previously defaulted to `extra='ignore'` and the comment notes they were silently dropped. The model defines `WasbIndexConfig` explicitly so config knobs take effect. I should add `stream_video` to `WasbIndexConfig` so the new knob is recognized and validated. Let me add it.

107. user (2026-06-15T16:47:42.867Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\agent-a771ff1d85e18f2ed\backend\config\models.py has been updated successfully. (file state is current in your context ? no need to Read it back)

108. assistant (2026-06-15T16:47:45.703Z)

Now let me check `test_config_wasb.py` to see if it asserts the exact field set (which would need updating) and run it.

109. user (2026-06-15T16:47:46.265Z)

```
1       """indexing.wasb.* and the other live indexing knobs must actually take effect.
2
3       Regression for the nested-config-severance finding: only AppConfig had extra='allow', so
4       nested sections silently dropped unknown keys and the whole indexing.wasb block (the
live
5       shuttle substrate's config) was discarded ? config.json overrides did nothing.
6       """
7       from backend.config.models import AppConfig, IndexingConfig
8       from backend.pipeline.detectors.wasb_runner import WasbConfig
9
10
11      def test_wasb_override_reaches_runner():
12          cfg = AppConfig(indexing={"wasb": {
13              "weights_path": "/custom/weights.pth", "velocity_threshold": 0.09, "conda_env":
"myenv",
14          }})
15          idx = cfg.get("indexing")
```

```
16          assert idx["wasb"]["weights_path"] == "/custom/weights.pth"
17          assert idx["wasb"]["velocity_threshold"] == 0.09
18          # And the runner's config builder actually picks it up (previously always
defaulted).
19          wcfg = WasbConfig.from_indexing_cfg(idx)
20          assert wcfg.weights_path == "/custom/weights.pth"
21          assert wcfg.conda_env == "myenv"
22
23
24      def test_live_indexing_knobs_declared():
25          fields = IndexingConfig.model_fields
26          assert "yolo_prefetch_workers" in fields
27          assert "debug_duration" in fields
28          assert "wasb" in fields
29          cfg = AppConfig(indexing={"yolo_prefetch_workers": 7, "debug_duration": 30})
30          idx = cfg.get("indexing")
31          assert idx["yolo_prefetch_workers"] == 7
32          assert idx["debug_duration"] == 30
33
34
35      def test_ai_chunk_scale_width_declared_and_overridable():
36          # Must be DECLARED (nested sections drop unknown keys) and default to 1280 ?
37          # the gemini segmenter re-encodes upload chunks at this width so 4K sources
38          # don't exceed the provider's upload cap (the silent 0-segments failure).
39          assert "ai_chunk_scale_width" in IndexingConfig.model_fields
40          assert AppConfig().get("indexing")["ai_chunk_scale_width"] == 1280
41          cfg = AppConfig(indexing={"ai_chunk_scale_width": 960})
42          assert cfg.get("indexing")["ai_chunk_scale_width"] == 960
43
44
45      def test_dead_knobs_removed():
46          fields = IndexingConfig.model_fields
47          assert "yolo_parallel_chunks" not in fields
48          assert "yolo_chunk_workers" not in fields
49
```

110. assistant (2026-06-15T16:47:49.652Z)

This test doesn't assert an exact field set, so adding `stream_video` won't break it. Let me add a small assertion that the new knob is declared and flows through (it's a config that must take effect, matching the file's theme). I'll extend `test_wasb_override_reaches_runner`.

111. user (2026-06-15T16:47:56.253Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\agent-a771ff1d85e18f2ed\tests\test_config_wasb.py has been updated successfully. (file state is current in your context ? no need to Read it back)

112. assistant (2026-06-15T16:47:59.986Z)

Now let me run the config, runner, and cache tests, plus the WASB runner test file and the broader detector tests, and mypy on the config model.

113. user (2026-06-15T16:48:09.178Z)

```
.....................................................F.........     [100%]
================================== FAILURES ===================================
_____________________ test_stage_video_success_and_failure _____________________

    def test_stage_video_success_and_failure():
        r = WasbRunner(WasbConfig())
        with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="OK")):
>           assert r._stage_video(r"C:\v\clip.mp4", "clip.mp4") == "~/clips/clip.mp4"
E           AssertionError: assert None == '~/clips/clip.mp4'
```

```
E            +  where None = _stage_video('C:\\v\\clip.mp4', 'clip.mp4')
E            +    where _stage_video = <backend.pipeline.detectors.wasb_runner.WasbRunner object
at 0x000002297EF348D0>._stage_video

tests\test_wasb_runner.py:42: AssertionError
------------------------------ Captured log call ------------------------------
ERROR    backend.pipeline.detectors.wasb_runner:wasb_runner.py:92 Cannot stage video into WSL:
video source is not readable from WSL at '/mnt/c/v/clip.mp4'. Google Drive, UNC, and other
network/virtual filesystems are invisible to WSL's /mnt ? copy the video to a local disk (e.g.
C:\ or the WSL filesystem) and retry.
========================== short test summary info ==========================
FAILED tests/test_wasb_runner.py::test_stage_video_success_and_failure - Asse...
1 failed, 67 passed in 1.20s
```

114. user (2026-06-15T16:48:15.298Z)

```
Success: no issues found in 3 source files
===ruff all changed===
All checks passed!
```

115. assistant (2026-06-15T16:48:20.311Z)

The 1 failure in `test_wasb_runner.py::test_stage_video_success_and_failure` is **pre-existing and unrelated to my change** ? it expects `_stage_video` to succeed with a single `_wsl_bash` returning "OK", but the readability preflight (added in commit #177) needs "READABLE" first. The `test_tracknet_runner.py` version of this test was already updated for the preflight (it uses `side_effect=[_proc(0, stdout="READABLE"), _proc(0, stdout="OK")]`), but `test_wasb_runner.py` wasn't. Let me confirm this fails on clean master (i.e., it's not my regression).

116. user (2026-06-15T16:48:22.523Z)

```
def test_healthcheck_no_wsl_binary():
    r = WasbRunner(WasbConfig())
    with patch.object(r, "_wsl_bash", side_effect=FileNotFoundError()):
        ok, msg = r.healthcheck()
    assert not ok and "wsl" in msg.lower()


def test_stage_video_success_and_failure():
    r = WasbRunner(WasbConfig())
    with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="OK")):
        assert r._stage_video(r"C:\v\clip.mp4", "clip.mp4") == "~/clips/clip.mp4"
    with patch.object(r, "_wsl_bash", return_value=_proc(1, stderr="No such file")):
        assert r._stage_video(r"C:\v\clip.mp4", "clip.mp4") is None


def test_run_predict_returns_none_when_infer_script_stage_fails(tmp_path):
    r = WasbRunner(WasbConfig())
    with patch.object(r, "_stage_infer_script", return_value=False):
        assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123") is None
```

--- cache hygiene (keep_frames / min_free_gb / shared WSL helpers) ----------

```
def test_from_indexing_cfg_reads_cleanup_knobs():
```

117. assistant (2026-06-15T16:48:26.049Z)

Confirmed: this test exists identically on `origin/master` and is a **pre-existing failure** introduced by the #177 readability-preflight change ? `test_wasb_runner.py` was never updated to match (its sibling `test_tracknet_runner.py` was). It is entirely unrelated to my change. Let me verify it fails on a clean checkout of master to be 100% certain.

118. user (2026-06-15T16:48:29.749Z)

```
Saved working directory and index state WIP on worktree-agent-a771ff1d85e18f2ed: 46ad601
fix(detectors): fail fast with an actionable error when a video source is unreadable from WSL
(#177)
tests\test_wasb_runner.py:42: AssertionError
------------------------------ Captured log call ------------------------------
ERROR    backend.pipeline.detectors.wasb_runner:wasb_runner.py:86 Cannot stage video into WSL:
video source is not readable from WSL at '/mnt/c/v/clip.mp4'. Google Drive, UNC, and other
network/virtual filesystems are invisible to WSL's /mnt ? copy the video to a local disk (e.g.
C:\ or the WSL filesystem) and retry.
=========================== short test summary info ===========================
FAILED tests/test_wasb_runner.py::test_stage_video_success_and_failure - Asse...
1 failed in 0.26s
=== restoring ===

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (8ff804df5ecdebc138e8f8dc20b11bd8dfdd254f)
```

119. assistant (2026-06-15T16:48:34.289Z)

Confirmed: the failure is **pre-existing on `origin/master`** (a stale test from PR #177's preflight change), completely independent of my work. It's out of scope for this focused perf PR, but I'll flag it as a background task.

Now let me run the full test suite to confirm no other regressions from my changes. Let me use the repo's runner.

120. user (2026-06-15T16:50:02.284Z)

```
        step later."""
        r = WasbRunner(WasbConfig())  # weights default lives under ~/models
        cmds = []

        def fake_wsl(cmd):
            cmds.append(cmd)
            if "wasb_infer.py" in cmd:
                # simulate the run producing the expected CSV
                (tmp_path / "clip__abcdefabcdef_wasb.csv").write_text("Frame,Visibility,X,Y\n")
            return _proc()

        with patch.object(r, "_wsl_bash", side_effect=fake_wsl):
            out = r.run_predict("C:\\vids\\clip.mp4", str(tmp_path),
video_id="abcdefabcdef0000")
>       assert out is not None
E       assert None is not None

tests\test_wsl_tilde_quoting.py:110: AssertionError
------------------------------ Captured log call ------------------------------
ERROR    backend.pipeline.detectors.wasb_runner:wasb_runner.py:92 Cannot stage video into WSL:
video source is not readable from WSL at '/mnt/c/vids/clip.mp4'. Google Drive, UNC, and other
network/virtual filesystems are invisible to WSL's /mnt ? copy the video to a local disk (e.g.
C:\ or the WSL filesystem) and retry.
=========================== short test summary info ===========================
FAILED tests/test_wasb_runner.py::test_stage_video_success_and_failure - Asse...
FAILED tests/test_wsl_tilde_quoting.py::test_wasb_stage_video_cmd_has_no_quoted_tilde
FAILED tests/test_wsl_tilde_quoting.py::test_wasb_run_predict_resolves_relative_out_dir
FAILED tests/test_wsl_tilde_quoting.py::test_wasb_run_predict_inference_cmd_has_no_quoted_tilde
4 failed, 698 passed, 7 skipped in 80.91s (0:01:20)
```

121. assistant (2026-06-15T16:50:06.339Z)

4 failures. Let me check whether these are all the same pre-existing readability-preflight issue (the `fake_wsl` mocks don't return "READABLE"). The error message for `test_wsl_tilde_quoting`

failures is the same "not readable from WSL" preflight error. Let me confirm all 4 fail on clean master.

122. user (2026-06-15T16:50:13.556Z)

```
Saved working directory and index state WIP on worktree-agent-a771ff1d85e18f2ed: 46ad601
fix(detectors): fail fast with an actionable error when a video source is unreadable from WSL
(#177)
------------------------------ Captured log call -------------------------------
ERROR    backend.pipeline.detectors.wasb_runner:wasb_runner.py:86 Cannot stage video into WSL:
video source is not readable from WSL at '/mnt/c/vids/clip.mp4'. Google Drive, UNC, and other
network/virtual filesystems are invisible to WSL's /mnt ? copy the video to a local disk (e.g.
C:\ or the WSL filesystem) and retry.
=========================== short test summary info ============================
FAILED tests/test_wasb_runner.py::test_stage_video_success_and_failure - Asse...
FAILED tests/test_wsl_tilde_quoting.py::test_wasb_stage_video_cmd_has_no_quoted_tilde
FAILED tests/test_wsl_tilde_quoting.py::test_wasb_run_predict_resolves_relative_out_dir
FAILED tests/test_wsl_tilde_quoting.py::test_wasb_run_predict_inference_cmd_has_no_quoted_tilde
4 failed in 0.86s
=== restoring ===
no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (6b185f49d9fcb34455be099d6e7b0854a9f14465)
```

123. assistant (2026-06-15T16:50:19.351Z)

All 4 failures are **pre-existing on `origin/master`** ? they're stale WASB-runner tests that PR #177's readability preflight broke (those tests mock `_wsl_bash` without returning "READABLE" for the preflight). They are unrelated to my changes. My changes add 0 new failures: the suite is `698 passed` with my changes vs the same 4 pre-existing failures on master.

Interestingly, my new `test_run_predict_streaming_*` tests in `test_tracknet_runner.py` correctly mock `_stage_video` directly (returning the staged path), so they sidestep the preflight and pass. The stale tests in the OTHER two files use a different mocking approach that #177 broke.

This stale-test breakage is a real pre-existing issue worth a separate fix. Let me flag it as a background task (out of scope for this perf PR), then commit my work.

124. user (2026-06-15T16:50:29.545Z)

Noted (position 1, task_id: task_fcb8ed61). A chip is showing for the user ? they can start it in a fresh worktree with one click, or dismiss it. If this suggestion becomes stale or superseded, call dismiss_task with this task_id. Currently pending: task_fcb8ed61 "Fix stale WASB runner tests broken by #177 preflight". Continue your current work.

125. assistant (2026-06-15T16:50:32.542Z)

Now let me do a final review of the complete `wasb_infer.py` changes to make sure everything is coherent, then commit. Let me read the full modified `run()` body and the streaming orchestration once more.

126. user (2026-06-15T16:50:34.356Z)

```
318         try:
319             yield store
320         finally:
321             cv2.imread = real_imread  # type: ignore[assignment]
322 
323 
324     # ------------------------------------------------------------------------ #
325     # Inference
326     # ------------------------------------------------------------------------ #
327     def run(frames, weights: str, out_csv: str, sport: str = "badminton",
328             limit: int = 0, batch_size: int = 8, num_workers: int = 4,
```

```
329             log_every_batches: int = 50, cache_dir: "str | None" = None,
330             flush_every: int = 1000, fresh: bool = False,
331             frame_loader=None, source_key: "str | None" = None) -> str:
332         """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
333 
334         Two equivalent ways to supply pixels (output is bit-identical between them):
335 
336         * **frames-on-disk** (default): ``frames`` is a directory path string; frames are
337           globbed (``*.png``/``*.jpg``) and ``ImageDataset`` reads each file via
``cv2.imread``.
338         * **streaming** (fast path): ``frames`` is a pre-built ordered list of (synthetic)
339           frame paths and ``frame_loader(index) -> BGR uint8 ndarray`` supplies the decoded
340           pixels in-memory. We scope a ``cv2.imread`` shim over the DataLoader pass so every
341           other line of ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the
detector
342           sees is identical to the PNG round-trip (PNG is lossless). Streaming forces
343           ``num_workers=0`` (the in-process shim + buffer can't cross worker processes).
344 
345         ``source_key`` overrides the cache-keying identity (defaults to the abspath of the
346         frames dir); the streaming path passes a stable ``video:<abspath>`` key so a resume
347         after reboot still matches.
348         """
349         import time
350         import torch
351         from torch.utils.data import DataLoader
352         from detectors import build_detector
353         from trackers import build_tracker
354         from dataloaders import build_img_transforms, build_seq_transforms
355         from dataloaders.dataset_loader import ImageDataset
356         from utils import Center
357 
358         streaming = frame_loader is not None
359 
360         cfg = _build_cfg(weights, sport)
361         frames_in = int(cfg["model"]["frames_in"])
362         input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
363         output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
364 
365         if streaming:
366             # `frames` is already the ordered list of (synthetic) frame paths.
367             if not isinstance(frames, (list, tuple)):
368                 raise SystemExit("streaming mode requires `frames` to be a list of frame
paths")
369             frames = list(frames)
370             frames_dir = source_key or "stream"
371         else:
372             frames_dir = frames
373             frames = sorted(glob.glob(osp.join(frames_dir, "*.png")) +
glob.glob(osp.join(frames_dir, "*.jpg")))
374         if limit:
375             frames = frames[:limit]
376         if len(frames) < frames_in:
377             where = frames_dir if not streaming else "video stream"
378             raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")
379 
380         # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
checkpoint).
381         samples = []
382         for window in build_windows(frames, frames_in):
383             annos = [{"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
for p in window]
384             samples.append({"frames": window, "annos": annos})
385         windows_total = len(samples)
386 
387         # ---- cache setup / resume decision ---------------------------------- #
```

```
388         if cache_dir is None:
389             cache_dir = default_cache_dir(out_csv)
390         os.makedirs(cache_dir, exist_ok=True)
391         det_jsonl, _ = _cache_paths(cache_dir)
392         cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
```

127. assistant (2026-06-15T16:50:42.009Z)

The implementation is coherent. One important correctness consideration about the streaming resume: when `num_workers=0` and the DataLoader iterates `remaining = samples[windows_done:]`, the first sample's window starts at frame index `windows_done`. The dataset reads frames `windows_done, windows_done+1, windows_done+2`. The store is primed at `start_index=windows_done`, and the `frame_loader` seeks via `CAP_PROP_POS_FRAMES`. This matches.

There's a subtle edge: with `batch_size > 1` and `num_workers=0`, the DataLoader still calls `__getitem__` sequentially per sample to assemble each batch, so reads remain in non-decreasing order. Good.

Let me also double check that `limit` works correctly in streaming ? `frames` is truncated to `limit`, and the `frame_loader` will only be asked for indices `< limit`, which are `< count_hint`. The store's `total` is `len(frames)` (post-limit), so out-of-range protection aligns. Good.

Now let me verify the whole file compiles and re-confirm lint/type one final time, then create the branch and commit.

128. user (2026-06-15T16:51:30.732Z)

```
syntax OK
.....................................................               [100%]
58 passed in 43.58s
```

129. assistant (2026-06-15T16:51:33.780Z)

All green. Now let me create the branch from `origin/master` explicitly, verify HEAD, stage only my changed files, and commit.

130. user (2026-06-15T16:51:38.205Z)

```
M       backend/config/models.py
M       backend/pipeline/detectors/wasb_infer.py
M       backend/pipeline/detectors/wasb_runner.py
M       tests/test_config_wasb.py
M       tests/test_tracknet_runner.py
M       tests/test_wasb_cache.py
branch 'perf/wasb-fast-frames' set up to track 'origin/master'.
Switched to a new branch 'perf/wasb-fast-frames'
perf/wasb-fast-frames
46ad601 fix(detectors): fail fast with an actionable error when a video source is unreadable
from WSL (#177)
```

131. assistant (2026-06-15T16:51:41.207Z)

HEAD is at origin/master's tip (46ad601), my modified files carried over. Let me verify the diff is only my intended changes, then stage explicit paths and commit.

132. user (2026-06-15T16:51:43.284Z)

```
 backend/config/models.py                  |   6 +
 backend/pipeline/detectors/wasb_infer.py  | 277 ++++++++++++++++++++++++++++++--
 backend/pipeline/detectors/wasb_runner.py |  26 ++-
 tests/test_config_wasb.py                 |  10 ++
 tests/test_tracknet_runner.py             |  43 +++++
```

```
 tests/test_wasb_cache.py                  | 143 +++++++++++++++
 6 files changed, 489 insertions(+), 16 deletions(-)
=== untracked ===
```

133. assistant (2026-06-15T16:51:47.067Z)

Only my 6 intended files changed, no untracked files to worry about (the `artifacts/` dir from the initial status isn't present in this worktree). Let me stage the explicit paths and commit.

134. user (2026-06-15T16:51:49.310Z)

```
M  backend/config/models.py
M  backend/pipeline/detectors/wasb_infer.py
M  backend/pipeline/detectors/wasb_runner.py
M  tests/test_config_wasb.py
M  tests/test_tracknet_runner.py
M  tests/test_wasb_cache.py
```

135. user (2026-06-15T16:52:08.535Z)

```
[perf/wasb-fast-frames 66bf0c6] perf(wasb): stream video frames to the detector instead of
extracting ~46k PNGs
 6 files changed, 489 insertions(+), 16 deletions(-)
66bf0c6 perf(wasb): stream video frames to the detector instead of extracting ~46k PNGs
```

136. assistant (2026-06-15T16:52:11.395Z)

Only my single commit is on the branch (no sibling commits). Now let me push and open the PR.

137. user (2026-06-15T16:52:14.461Z)

```
remote:
remote: Create a pull request for 'perf/wasb-fast-frames' on GitHub by visiting:
remote:      https://github.com/avidullu/badminton-highlight-indexer/pull/new/perf/wasb-fast-
frames
remote:
branch 'perf/wasb-fast-frames' set up to track 'origin/perf/wasb-fast-frames'.
To https://github.com/avidullu/badminton-highlight-indexer.git
 * [new branch]      perf/wasb-fast-frames -> perf/wasb-fast-frames
```

138. user (2026-06-15T16:52:49.963Z)

```
https://github.com/avidullu/badminton-highlight-indexer/pull/178
```

139. assistant (2026-06-15T16:52:53.181Z)

```
PR created: https://github.com/avidullu/badminton-highlight-indexer/pull/178
```

Let me do a final confirmation that the PR is open (not merged) and the branch contains only my commit.

140. user (2026-06-15T16:52:56.472Z)


```
{"baseRefName":"master","headRefName":"perf/wasb-fast-
frames","mergeable":"MERGEABLE","number":178,"state":"OPEN","title":"perf(wasb): stream video
frames to the detector instead of extracting ~46k PNGs"}
```


141. assistant (2026-06-15T16:53:11.663Z)

Done. PR #178 is OPEN against `master`, not merged.

Summary

Approach chosen: Option (a) ? stream frames directly from the video into the detector, eliminating the PNG disk round-trip, gated behind a default-OFF config flag.

The bottleneck was `wasb_infer.extract_frames`, which wrote ~46k PNGs for a 12-min 1080p60 clip (cv2 PNG encode + decode-on-read dominates; a real run overran the 1800s timeout and silently produced "0 rallies"). The fast path decodes the staged video on the fly with one forward-only `cv2.VideoCapture` and feeds frames into the detector via a scoped `cv2.imread` shim.

Why output is preserved exactly: PNG is lossless, so `cv2.imread(cv2.imwrite_png(frame))` returns the identical uint8 BGR array `cap.read()` produced. The shim only replaces the *source of pixels*; every other line of WASB's `ImageDataset.__getitem__` (affine resize to 512×288, ImageNet normalize, channel order) runs unchanged ? the detector sees the same frames, same order, same resolution, and the tensor is byte-for-byte identical to the disk path. The resumable cache, tracker re-run, and CSV writer are shared code.

Files changed (all absolute under the worktree):
- `backend/pipeline/detectors/wasb_infer.py` ? streaming support in `run()` (optional `frame_loader`/`source_key`, forces `num_workers=0`, `video:<abspath>` cache key); new pure helpers `synthetic_frame_path`, `build_windows`, `SequentialFrameStore` (bounded sliding buffer), the `_streaming_imread_patch` context manager, and `make_video_frame_loader`/`run_video_streaming` + `--stream-video` CLI flag.
- `backend/pipeline/detectors/wasb_runner.py` ? default-OFF `stream_video` config; passes `--stream-video` vs `--frames_out_dir`, skips disk guard and frame-cache cleanup when streaming.
- `backend/config/models.py` ? `stream_video` field on `WasbIndexConfig`.
- `tests/test_wasb_cache.py`, `tests/test_tracknet_runner.py`, `tests/test_config_wasb.py` ? unit tests for the pure helpers, eviction/resume semantics, the imread shim, and runner/config wiring (no torch/GPU/real-video needed).

Test results (local; GPU/WSL treated as unavailable): ruff clean, mypy clean on the 3 changed backend files, 58 passed across the touched suites + import smoke. A non-trivial design correction surfaced during testing ? the detector's per-window access dips back by `frames_in-1` at each window boundary, so the buffer evicts by `max_seen-(frames_in-1)`, not the naive max index.

Pre-existing failures (NOT mine): 4 stale WASB-runner test mocks break on `origin/master` from PR #177's readability preflight (verified by stashing my diff). Flagged as a separate background task; out of scope for this perf PR.

PR: https://github.com/avidullu/badminton-highlight-indexer/pull/178 ? left OPEN. It includes a "GPU verification + side-by-side plan" for the owner (diff the streamed vs PNG-path trajectory CSVs; expect identical) and notes CI is red account-wide due to the GitHub Actions billing outage, so verification was local. Not merged.